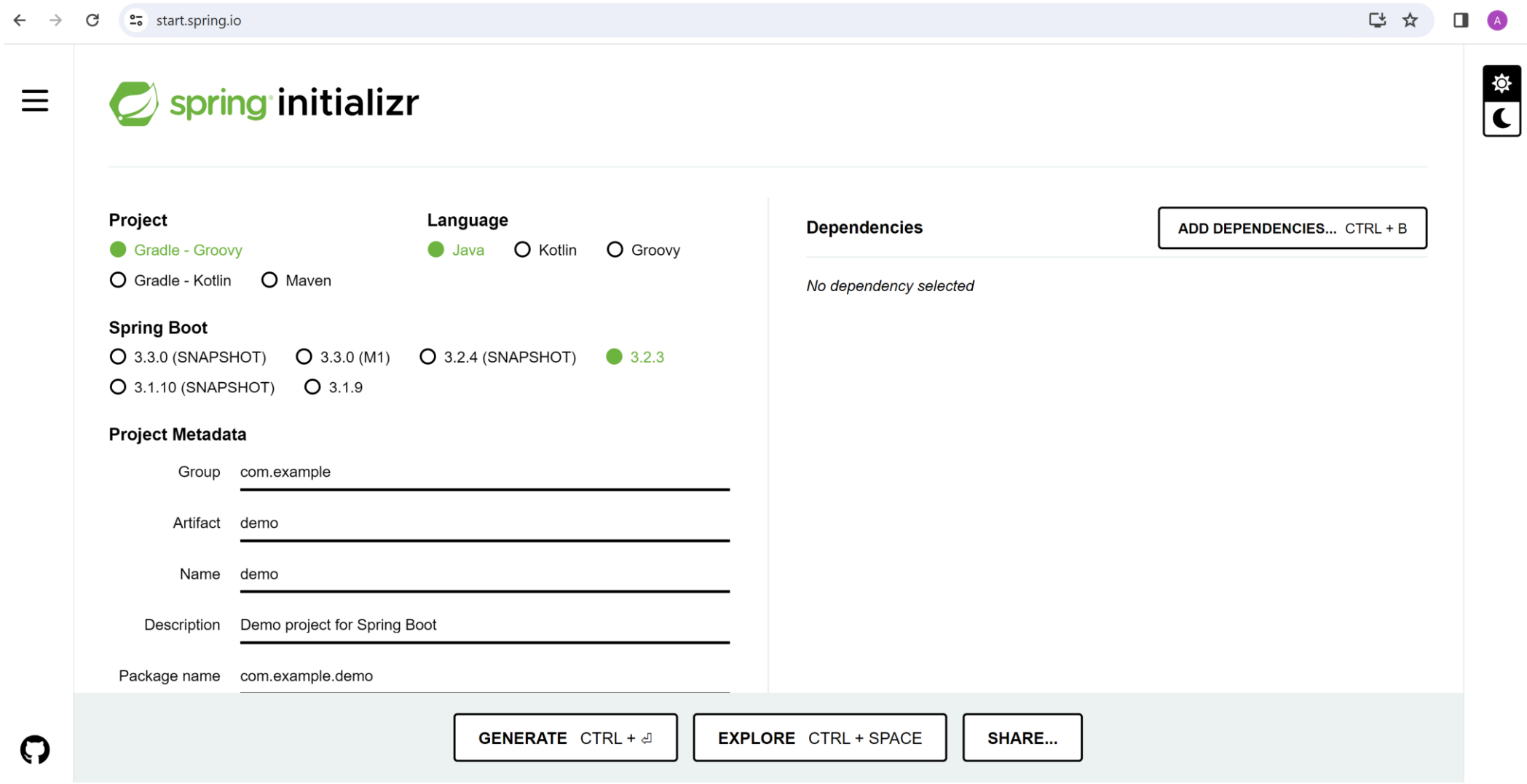
Web App Security
Basic Auth

arnaud.nauwynck@gmail.com

Practical Study :
Using No Code / Low Code

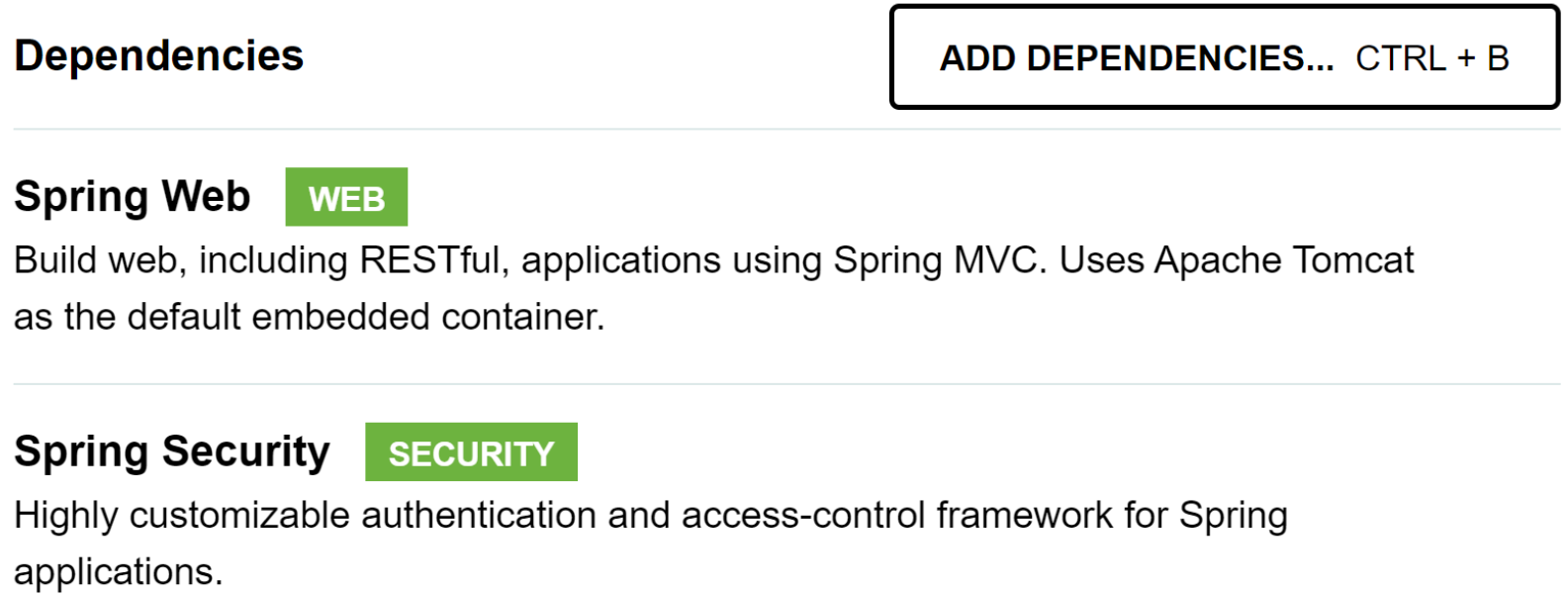
Java SpringBoot Security

StringInitializr [1/4]
Setup project: https://start.spring.io/

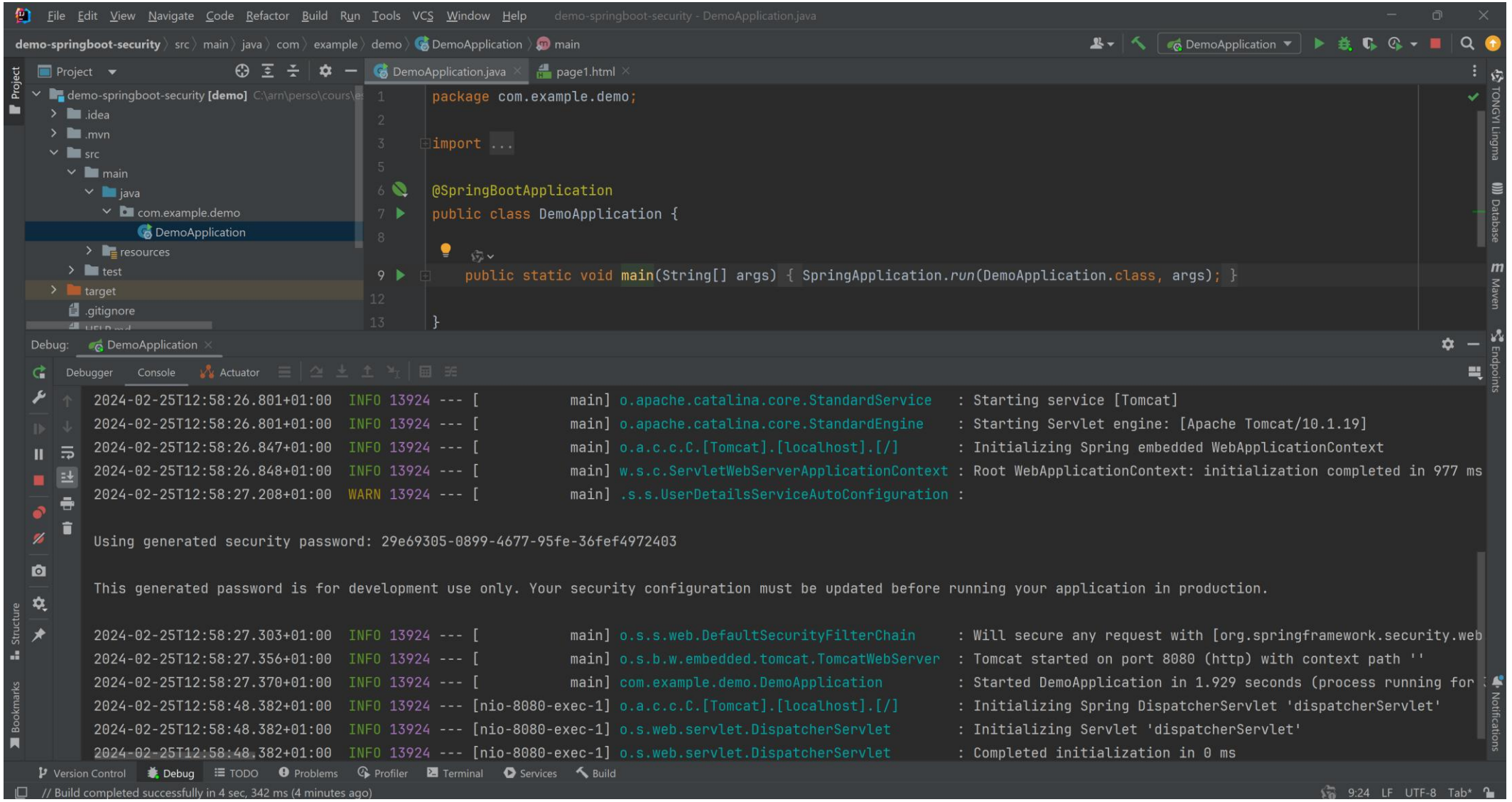


SpringInitializr [2/4]

Change to Maven
Enter Dependencies: Web, Security
Click "Generate" (Download zip)



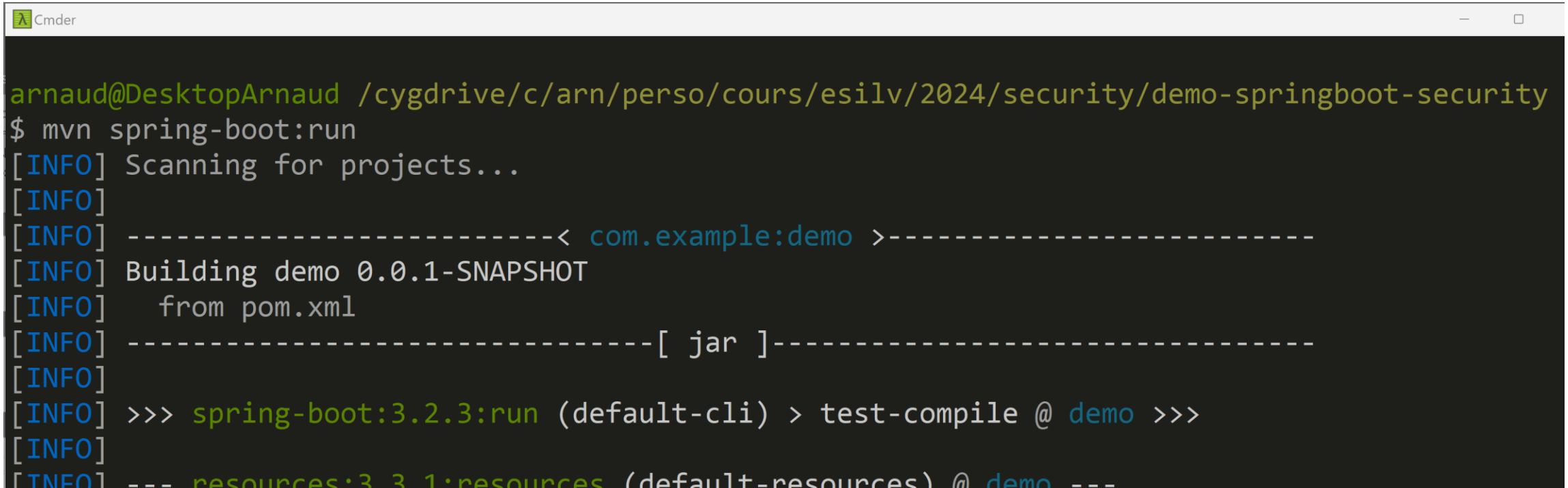
SpringInitializr [3/4]
Compile & Launch Web Server



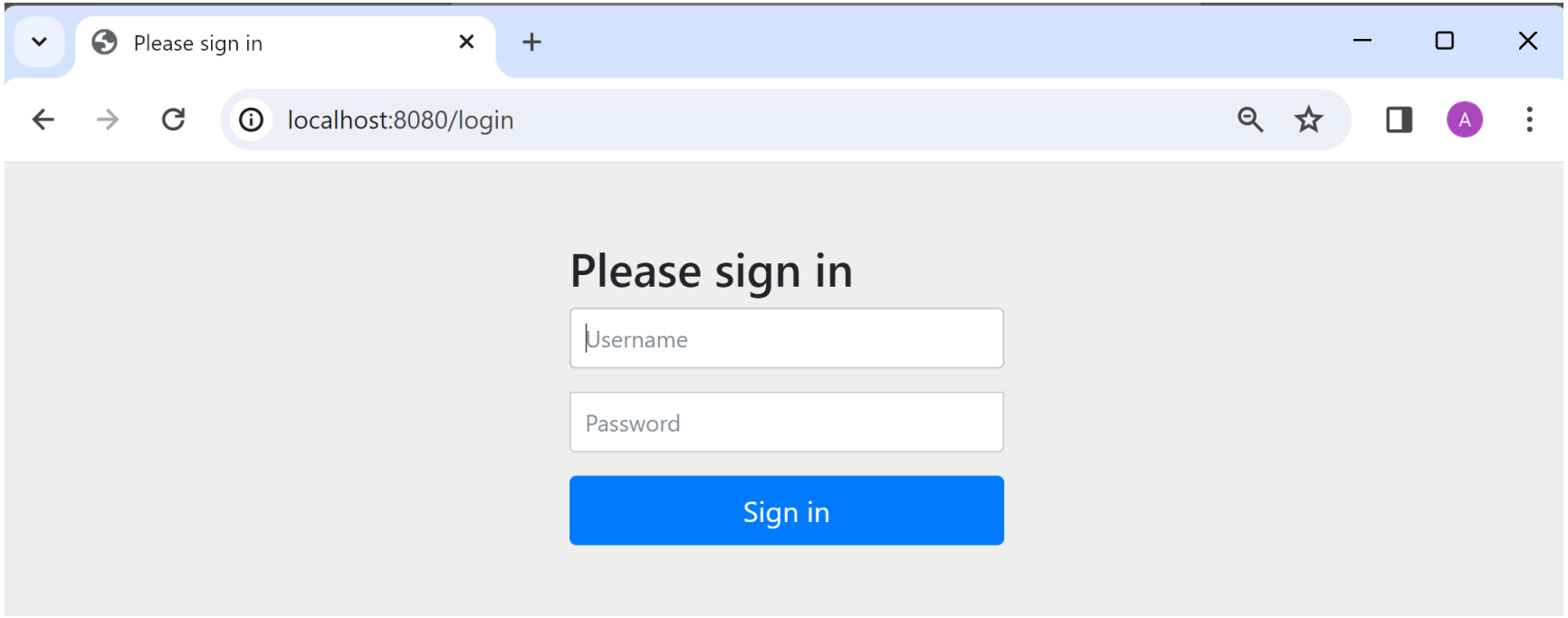
SpringInitializr [3/4]
Alternative .. Compile & Launch Web Server using maven

$ mvn package

$ mvn spring-boot:run



SpringInitializr [4/4]
Test: open http://localhost:8080



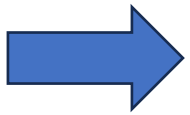
username = "user"
password = randomly generated at startup

```
2024-02-25T13:35:55.221+01:00  WARN 13900 --- [           main] .s.s.UserDetailsServiceAutoConfiguration :

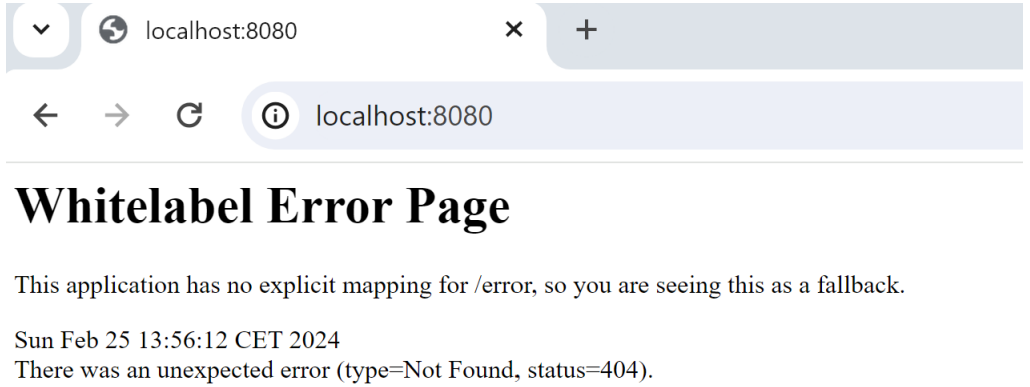
Using generated security password: 32b43163-eb64-428b-b616-094903ae17e6

This generated password is for development use only. Your security configuration must be updated before running your application in production.

2024-02-25T13:35:55.543+01:00  INFO 13900 --- [           main] o.s.s.web.DefaultSecurityFilterChain     : Will secure any request with [org.springframework.security.web
2024-02-25T13:35:55.716+01:00  INFO 13900 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (http) with context path ''
```

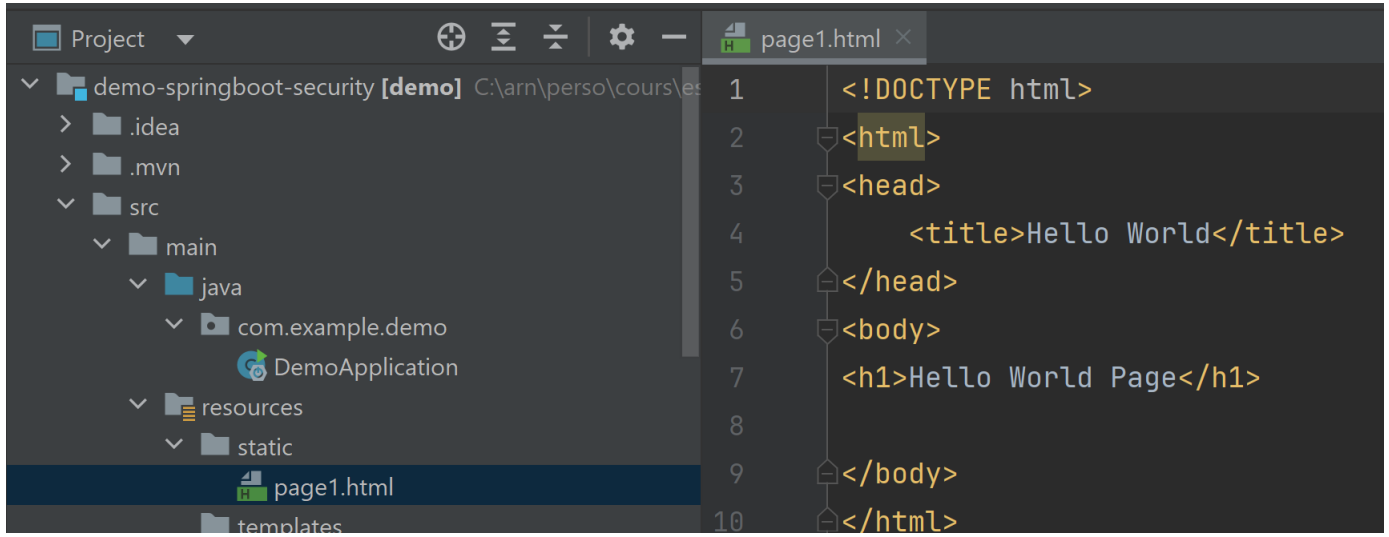


"OK" but 404
(no more 401)



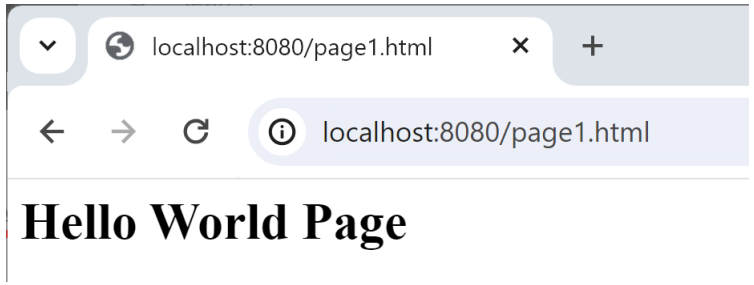
Add Resource static/page1.html
http GET /page1.html => Http 200

add file
src/main/resource/
static/page1.html



relaunch

http GET **/page1.html**



Install Demo & Hands-on

then pause

Detailed Analysis of 4 http scenarios

Scenario 1/4 : No authentication

Scenario 2/4 : BAD user/password

Scenario 3/4 : user/password OK => auth success, redirect, set Cookie

Scenario 4/4 : clear cookie

Detailed Analysis of 4 http scenarios



Scenario 1/4 : No authentication

Scenario 2/4 : BAD user/password

Scenario 3/4 : user/password OK => auth success, redirect, set Cookie

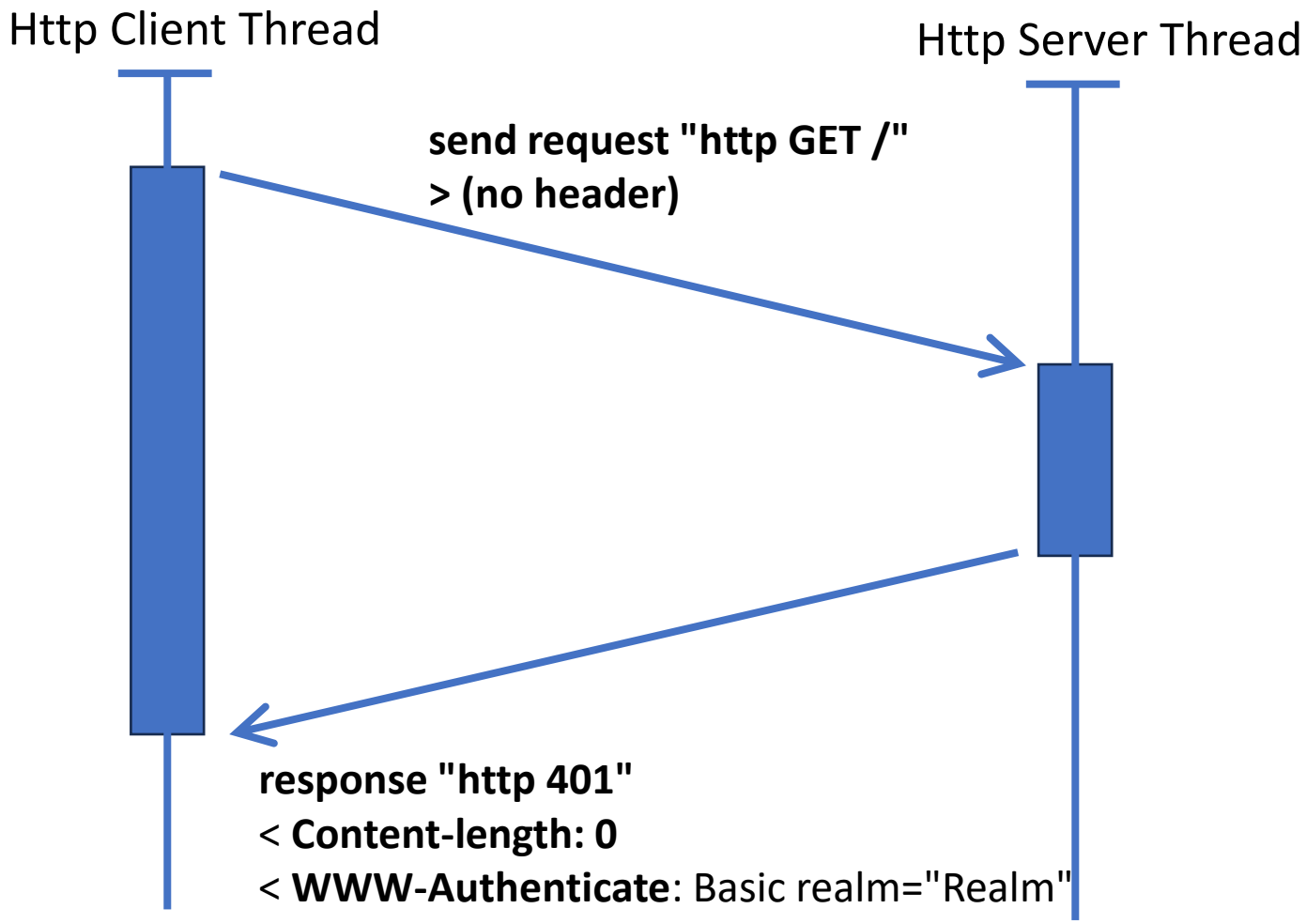
Scenario 4/4 : clear cookie

Scenario 1: NO authentication using curl

Cmder

```
$ curl -vv http://localhost:8080
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 401
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Set-Cookie: JSESSIONID=F4AC86090C48019BB14C4315942710AA; Path=/; HttpOnly
< WWW-Authenticate: Basic realm="Realm"
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< X-Frame-Options: DENY
< Content-Length: 0
< Date: Sun, 25 Feb 2024 12:36:32 GMT
<
* Connection #0 to host localhost left intact
```

Http Request/Response Sequence



401 : Unauthorized

MDN Web Docs
https://developer.mozilla.org › Web › HTTP › Status

401 Unauthorized - HTTP - MDN Web Docs

Apr 10, 2023 — The HyperText Transfer Protocol (**HTTP**) **401** Unauthorized response status code indicates that the client request has not been completed ...

403 Forbidden · WWW-Authenticate · 402 Payment Required

curl, no authentication data
=> NO result, only 401 response code

retry almost "Same" request in Web Browser
=> get different response !

http 301 ... redirect to login html page

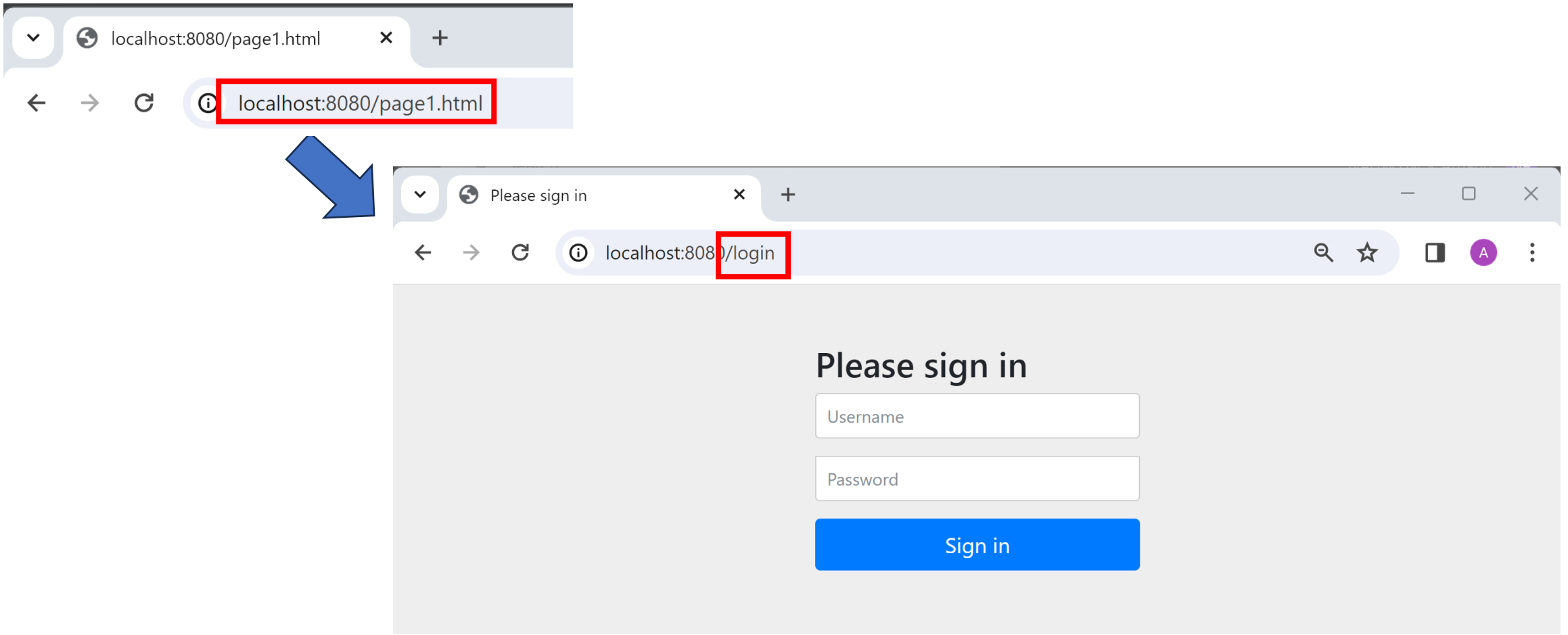
exactly "same" as Web Browser:
using curl to accept html content format... cf next

curl -H Accept:application/xhtml

Cmder

```
$ curl -vv http://localhost:8080/page1.html -H Accept:text/html,application/xhtml
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /page1.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.4.0
> Accept:text/html,application/xhtml
>
< HTTP/1.1 302
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Set-Cookie: JSESSIONID=3AF9670D9ACFF604C58140D4BA8159FB; Path=/; HttpOnly
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< X-Frame-Options: DENY
< Location: http://localhost:8080/login
< Content-Length: 0
< Date: Sun, 25 Feb 2024 17:22:50 GMT
<
* Connection #0 to host localhost left intact
```

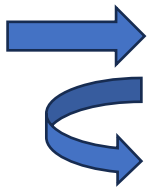
Scenario 1 .. redo using Web browser Chrome

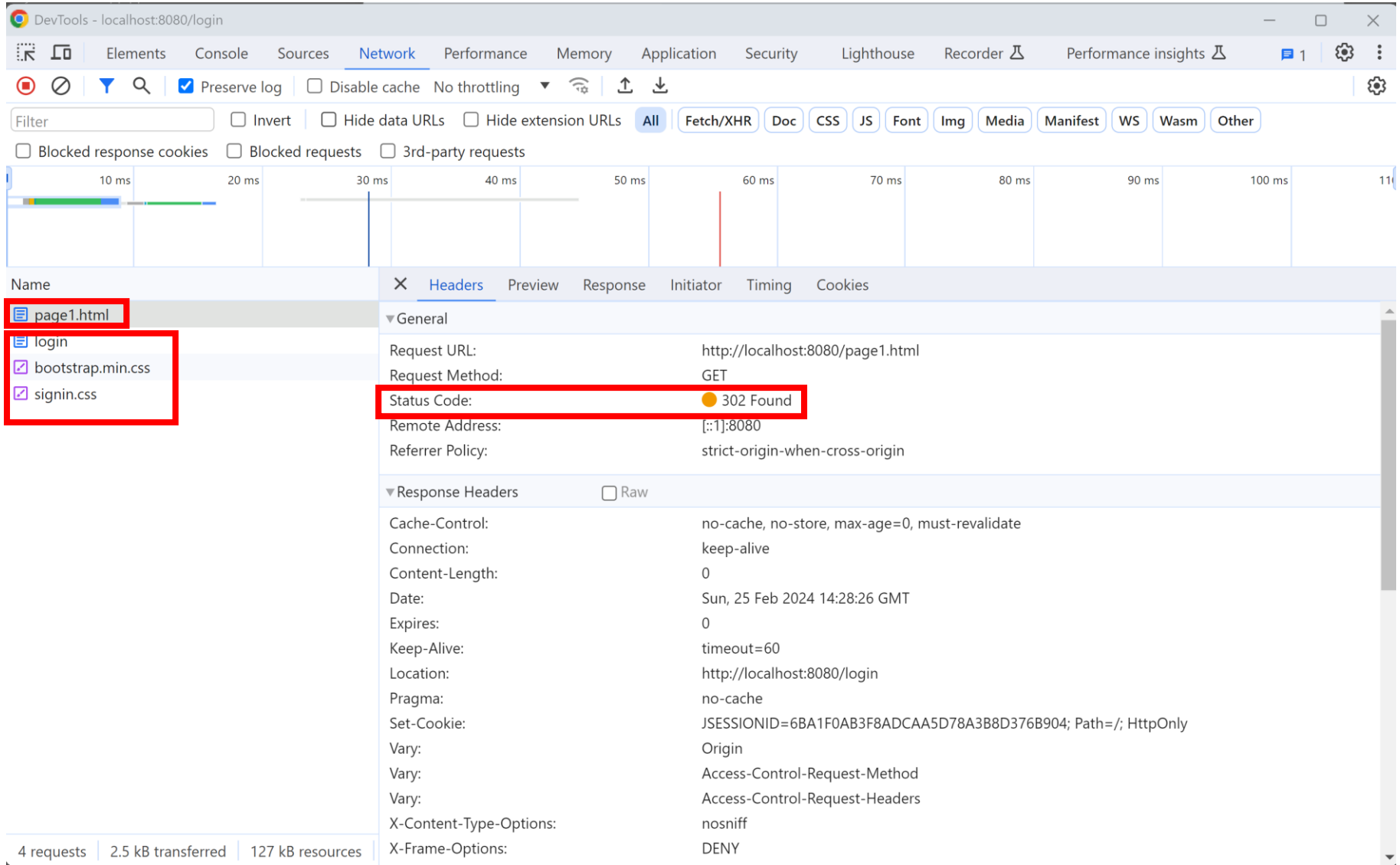


open (F12) : Developer Tools > Network "preserve log"=true

request
"/page1.html"

redirect "/login"





Detailed Request Headers

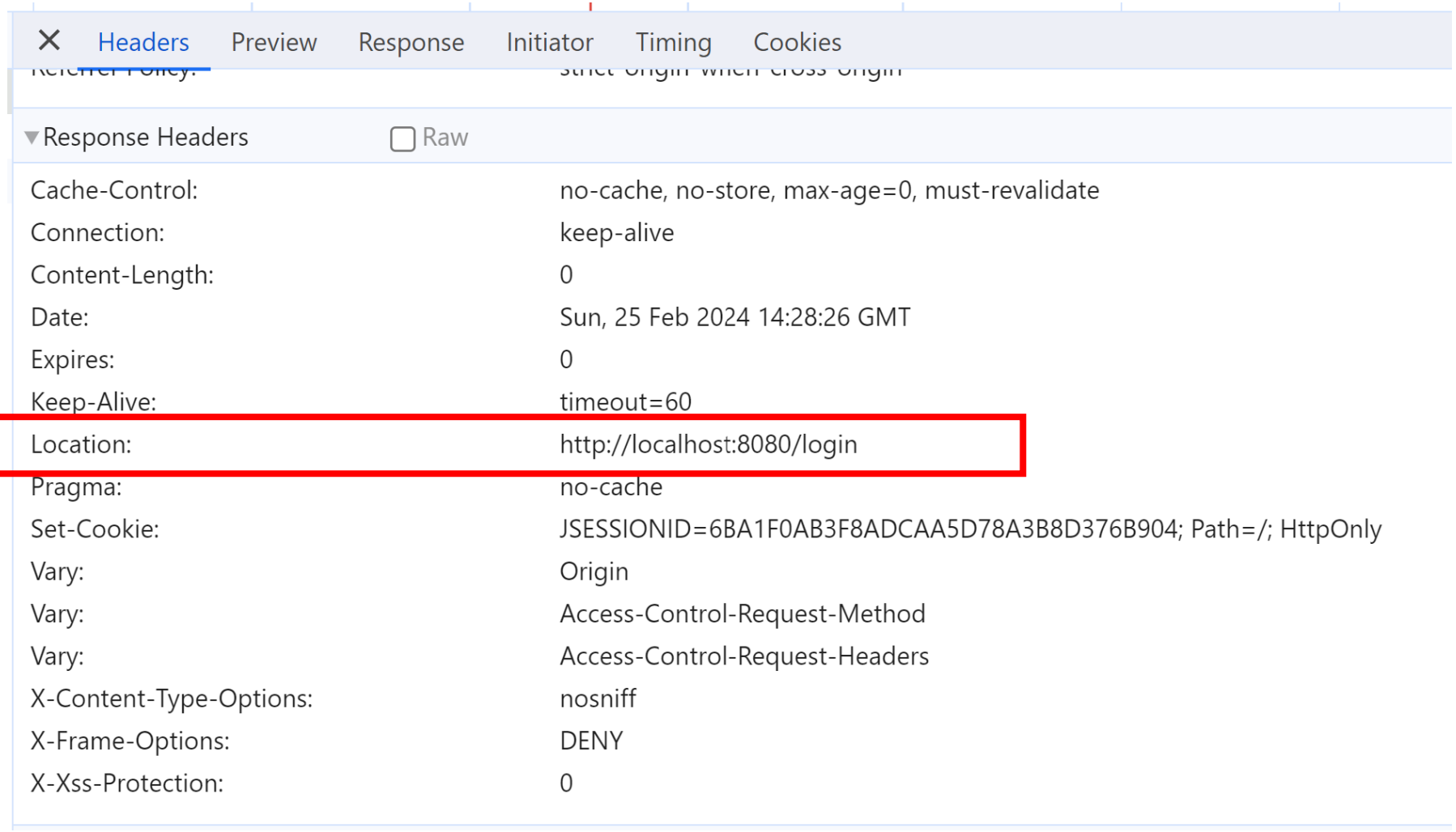
Headers | Preview | Response | Initiator | Timing | Cookies

X-Frame-Options:	DENY
X-Xss-Protection:	0

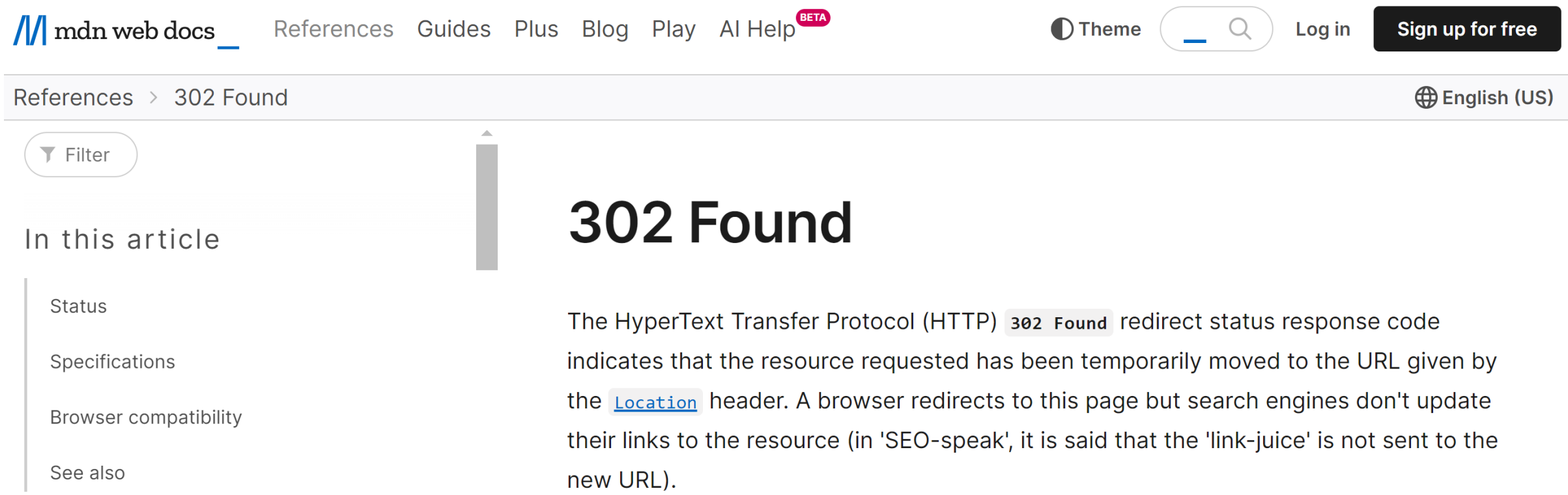
▼ Request Headers ☐ Raw

Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding:	gzip, deflate, br, zstd
Accept-Language:	en,fr-FR;q=0.9,fr;q=0.8,en-FR;q=0.7,en-US;q=0.6
Cache-Control:	max-age=0
Connection:	keep-alive
Cookie:	metabase.DEVICE=7856a6fd-dd9c-495d-880e-0b2ed106aab5; _ga=GA1.1.576560884.1694723966; JSESSIONID=41795E3A8958BD0E3F603879725681A0
Host:	localhost:8080
Sec-Ch-Ua:	"Not A(Brand";v="99", "Google Chrome";v="121", "Chromium";v="121"
Sec-Ch-Ua-Mobile:	?0
Sec-Ch-Ua-Platform:	"Windows"
Sec-Fetch-Dest:	document
Sec-Fetch-Mode:	navigate
Sec-Fetch-Site:	none
Sec-Fetch-User:	?1
Upgrade-Insecure-Requests:	1
User-Agent:	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36

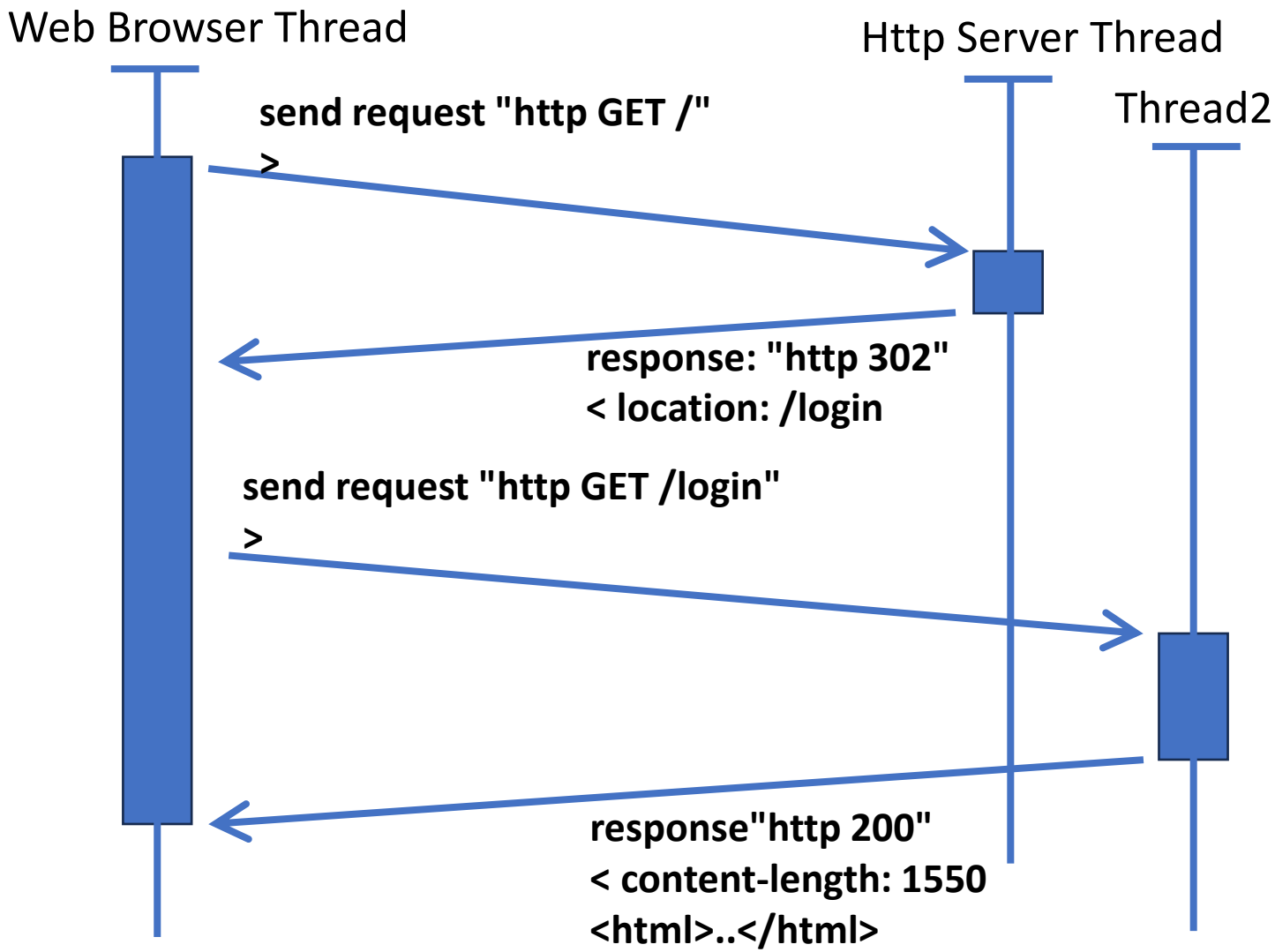
Detailed Response Headers



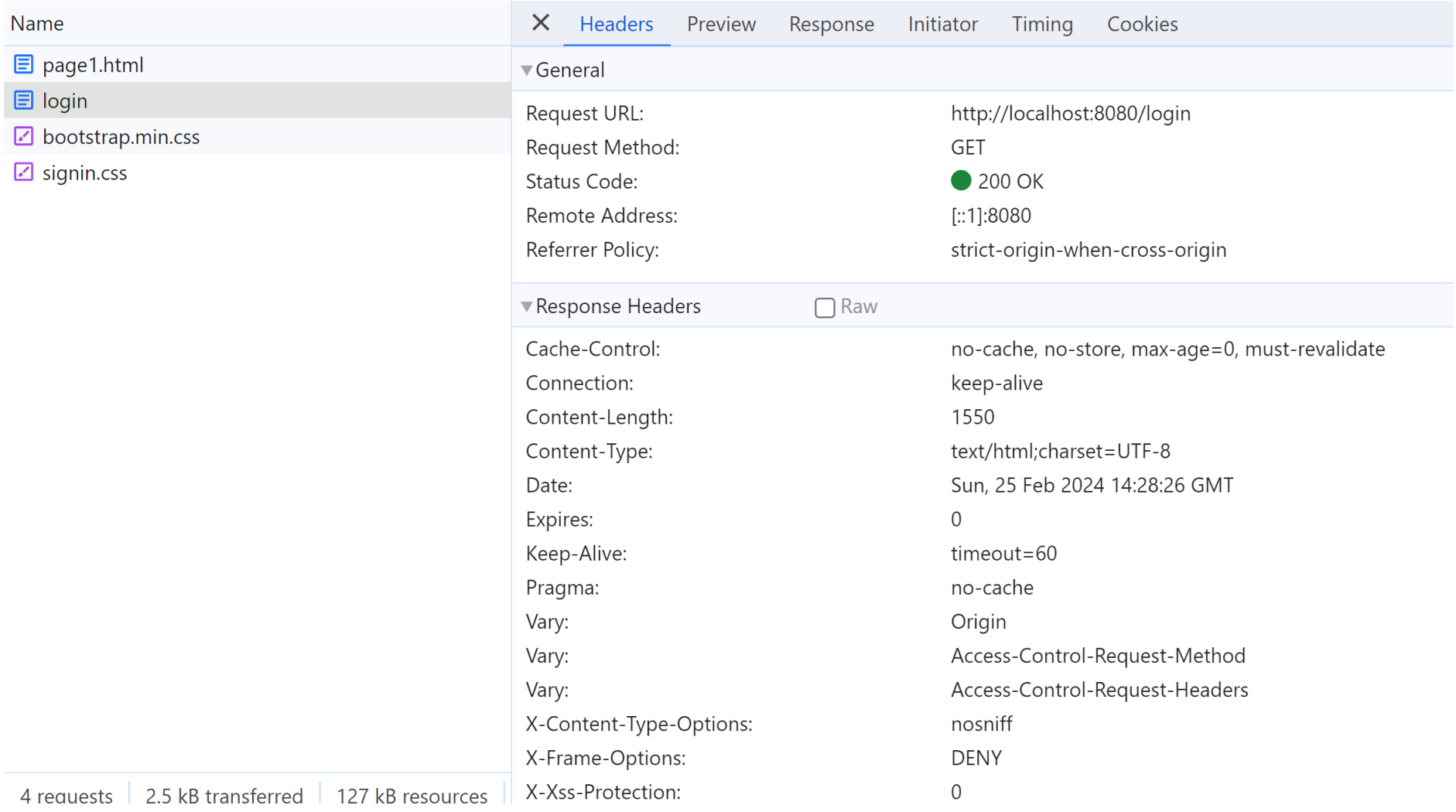
Http response code 302
+ Http Response Header "location:"
=> redirect



Http Redirect Sequence



Next request (after redirected)
GET /login => http 200 <html>..</html>



Detailed Analysis of 4 http scenarios

Scenario 1/4 : No authentication



Scenario 2/4 : BAD user/password

Scenario 3/4 : user/password OK => auth success, redirect, set Cookie

Scenario 4/4 : clear cookie

Wrong Password
401 : Unauthorized

```
$ curl -vv http://localhost:8080/page1.html -u user:WRONGPASS
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* Server auth using Basic with user 'user'
> GET /page1.html HTTP/1.1
> Host: localhost:8080
> Authorization: Basic dXNlcjpXUk9OR1BBU1M=
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 401
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
* Authentication problem. Ignoring this.
< WWW-Authenticate: Basic realm="Realm"
< X-Content-Type-Options: nosniff
```

Wrong User
401 : Unauthorized

Cmder

```
$ curl -vv http://localhost:8080/page1.html -u WRONGUSER:${PASSWORD}
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* Server auth using Basic with user 'WRONGUSER'
> GET /page1.html HTTP/1.1
> Host: localhost:8080
> Authorization: Basic V1JPTkdVU0VSOjMyYjQzMTYzLWViNjQtNDI4Yi1iNjE2LTA5
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 401
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
* Authentication problem. Ignoring this.
< WWW-Authenticate: Basic realm="Realm"
< X-Content-Type-Options: nosniff
```

All 401 !
NO Difference between

No Auth / Wrong User / Wrong Password

Rule of Thumb : never give any information (to attackers)

Detailed Analysis of 4 http scenarios

Scenario 1/4 : No authentication

Scenario 2/4 : BAD user/password



Scenario 3/4 : user/password OK => auth success, set Cookie

Scenario 4/4 : clear cookie

http GET using login/password
=> OK + get body

curl -vv http://localhost:8080/page1.html **-u user:${PASSWORD}**

Cmder

```
arnaud@DesktopArnaud /cygdrive/c/arn/devPerso
$ curl -vv http://localhost:8080/page1.html -u user:${PASSWORD}
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
* Server auth using Basic with user 'user'
> GET /page1.html HTTP/1.1
> Host: localhost:8080
> Authorization: Basic dXNlcjozMmI0MzE2My1lYjY0LTQyOGItYjYxNi0wOTQ5MDNhZTE3ZTY=
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 200
< Vary: Origin
```

Cmder

```
< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Last-Modified: Sun, 25 Feb 2024 11:57:53 GMT
< Accept-Ranges: bytes
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< X-Frame-Options: DENY
< Content-Type: text/html
< Content-Length: 61
< Date: Sun, 25 Feb 2024 14:19:45 GMT
<
<html>
<body>
<H1>Hello World Page</H1>

</body>
</html>* Connection #0 to host localhost left intact
```

empty line
(end of http headers)

61 bytes of data
then close TCP socket

Notice: curl -u user:${password}
> Authorization: Basic dXNlcjpwYXNzd29yZA==

user:password is sent as Http Header "Authorization: Basic <<base64>>"
in **CLEAR TEXT** (encoded in base 64)

to decode, use "base64 -d"



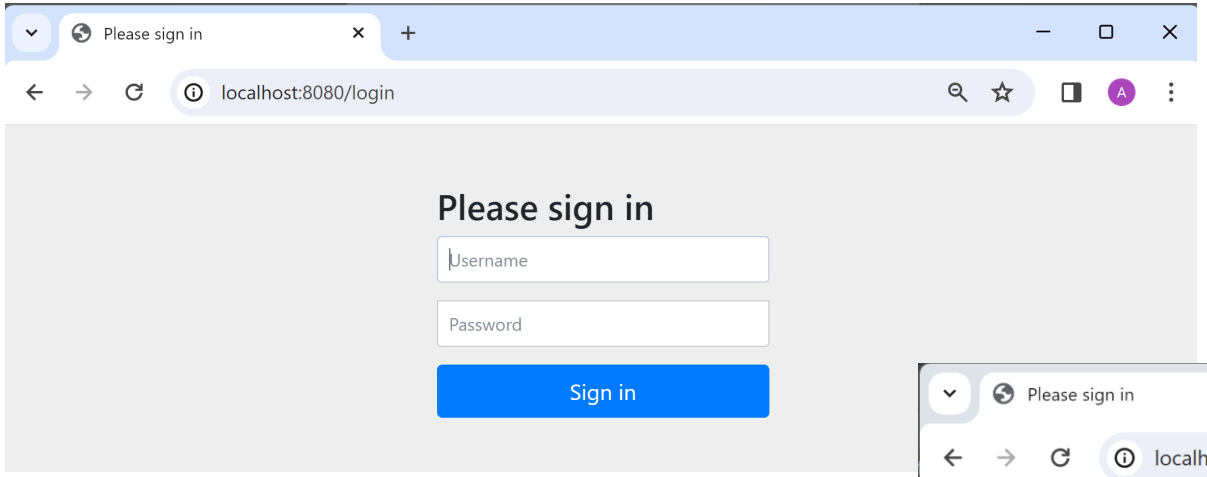
Remark 2 : when using explicit user:password => NO set-cookie

By default, Web Browsers do not send "user:password"

Server detect non interactive (batch) usage / Rest Api call

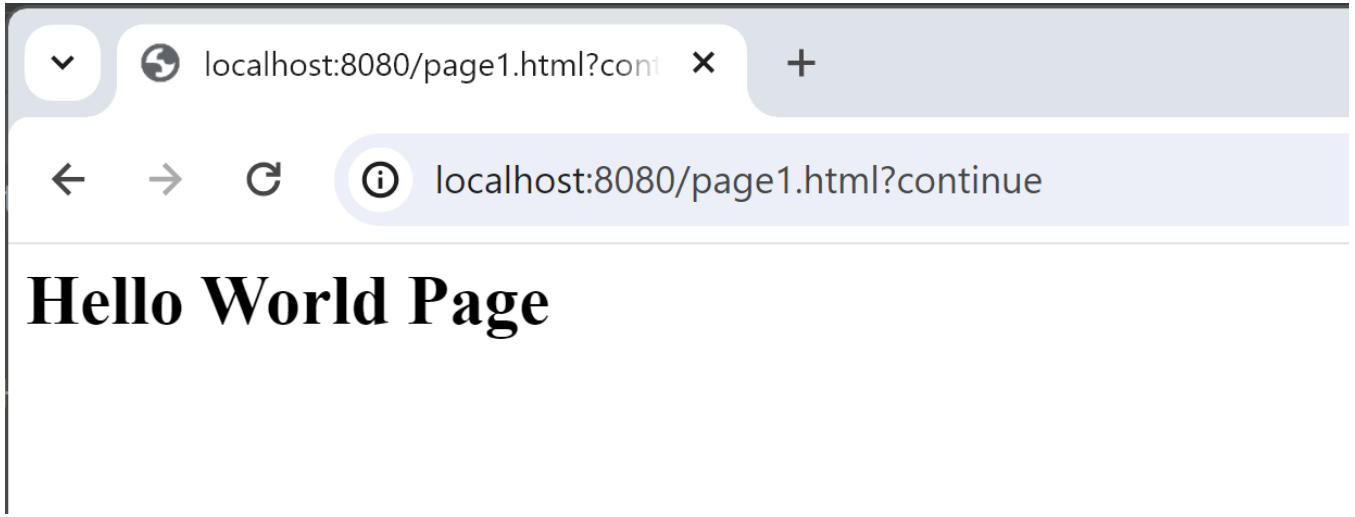
NO need to create interactive "session", so no set-cookie needed

user/password Using applicative Login Html page + Form + Button

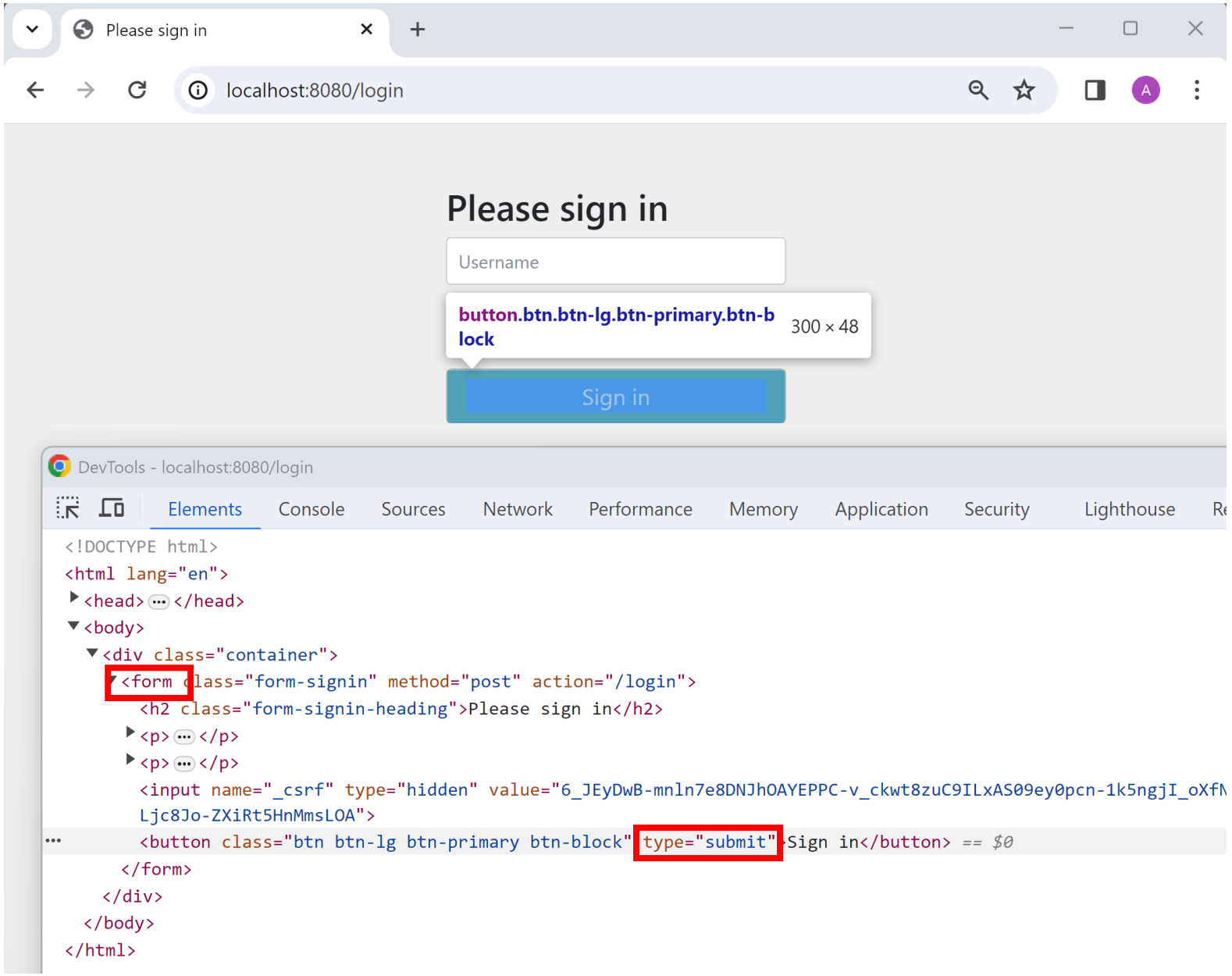




Authenticated OK => http 200 /page1.html?continue



F12 - Elements : Button Html Source



Details on <form>

```
<form class="form-signin" method="post" action="/login"> == $0
  <h2 class="form-signin-heading">Please sign in</h2>
  <p>
    <label for="username" class="sr-only">Username</label>
    <input type="text" id="username" name="username" class="form-control" placeholder="Username" required autofocus>
  </p>
  <p>
    <label for="password" class="sr-only">Password</label>
    <input type="password" id="password" name="password" class="form-control" placeholder="Password" required>
  </p>
  <input name="_csrf" type="hidden" value="6_JEyDwB-mnln7e8DNJhOAYEPPC-v_ckwt8zuC9ILxAS09ey0pcn-1k5ngjI_oXfNf9VCmc8E
  -ZXiRt5HnMmsLOA">
  <button class="btn btn-lg btn-primary btn-block" type="submit">Sign in</button>
</form>
```

<form> ... behavior explanation

on click button "Signin"

=> submit Form

=> call http POST "/login"
with payload = FormData { username, password, _csrf }

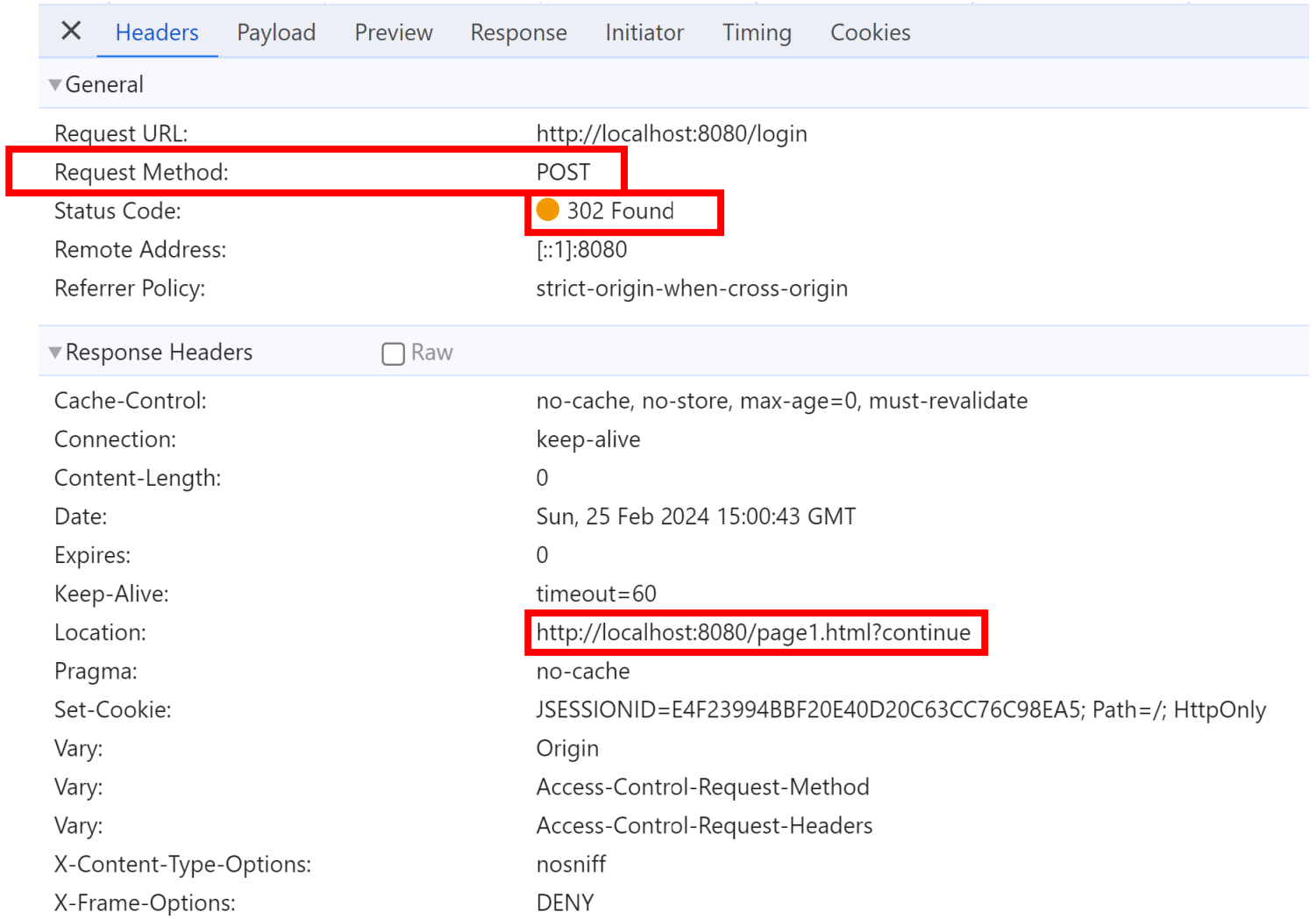
Notice: _csrf ?

C.S.R.F. stands for "Cross Site Request Forgery"

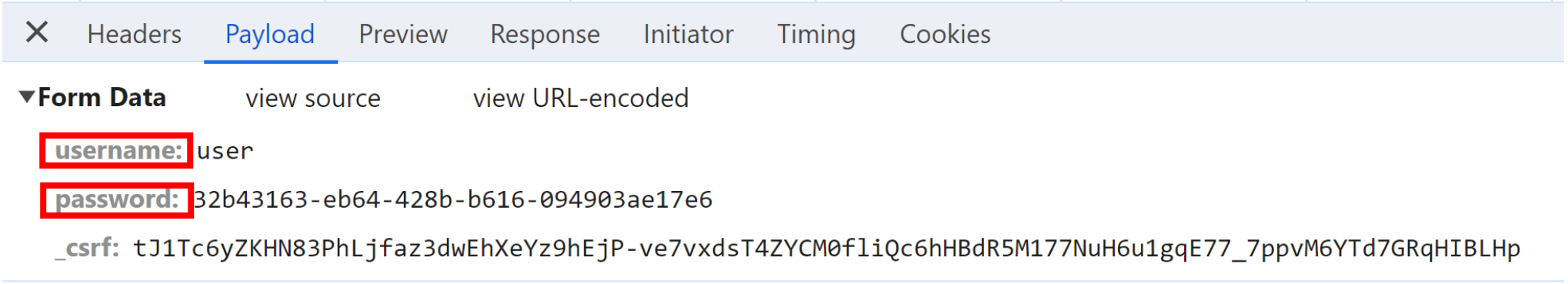
For preventing "http://a-malicious-site/click-me-page" to automatically call your http POST "/login" on your site and filling sensitive data from your browser

cf Appendix

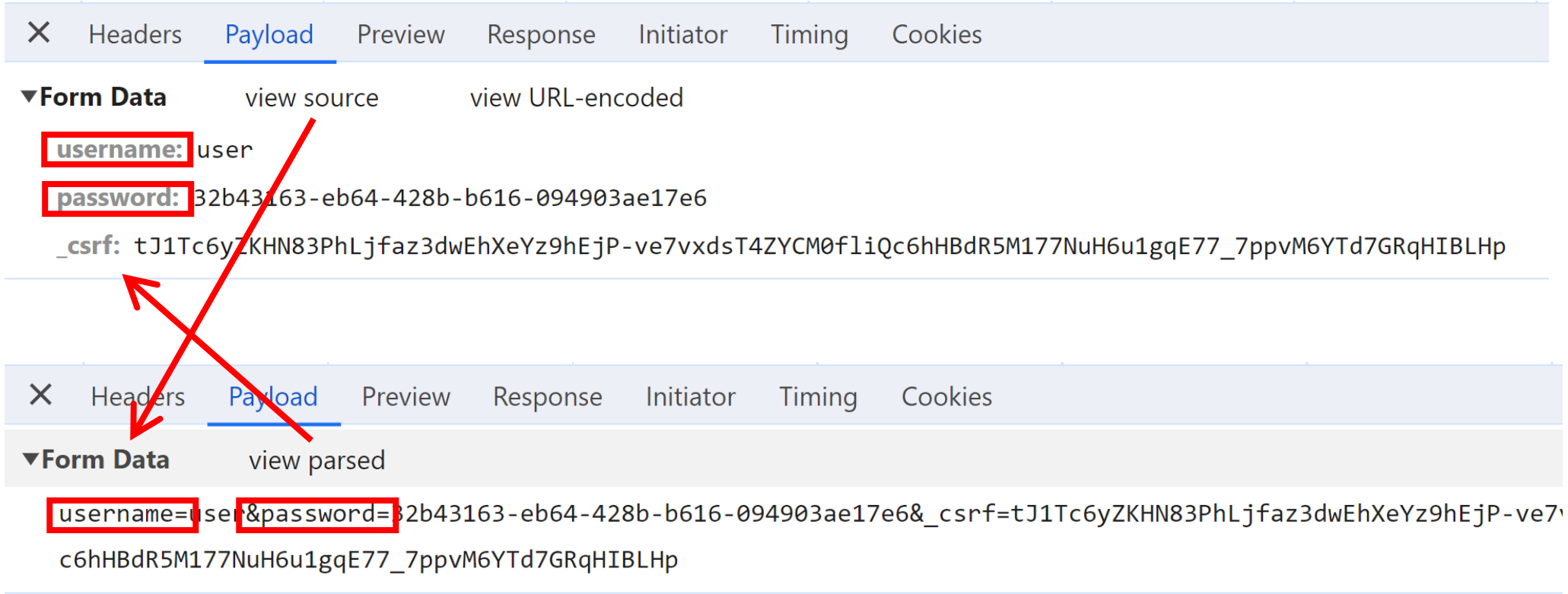
Http POST /login



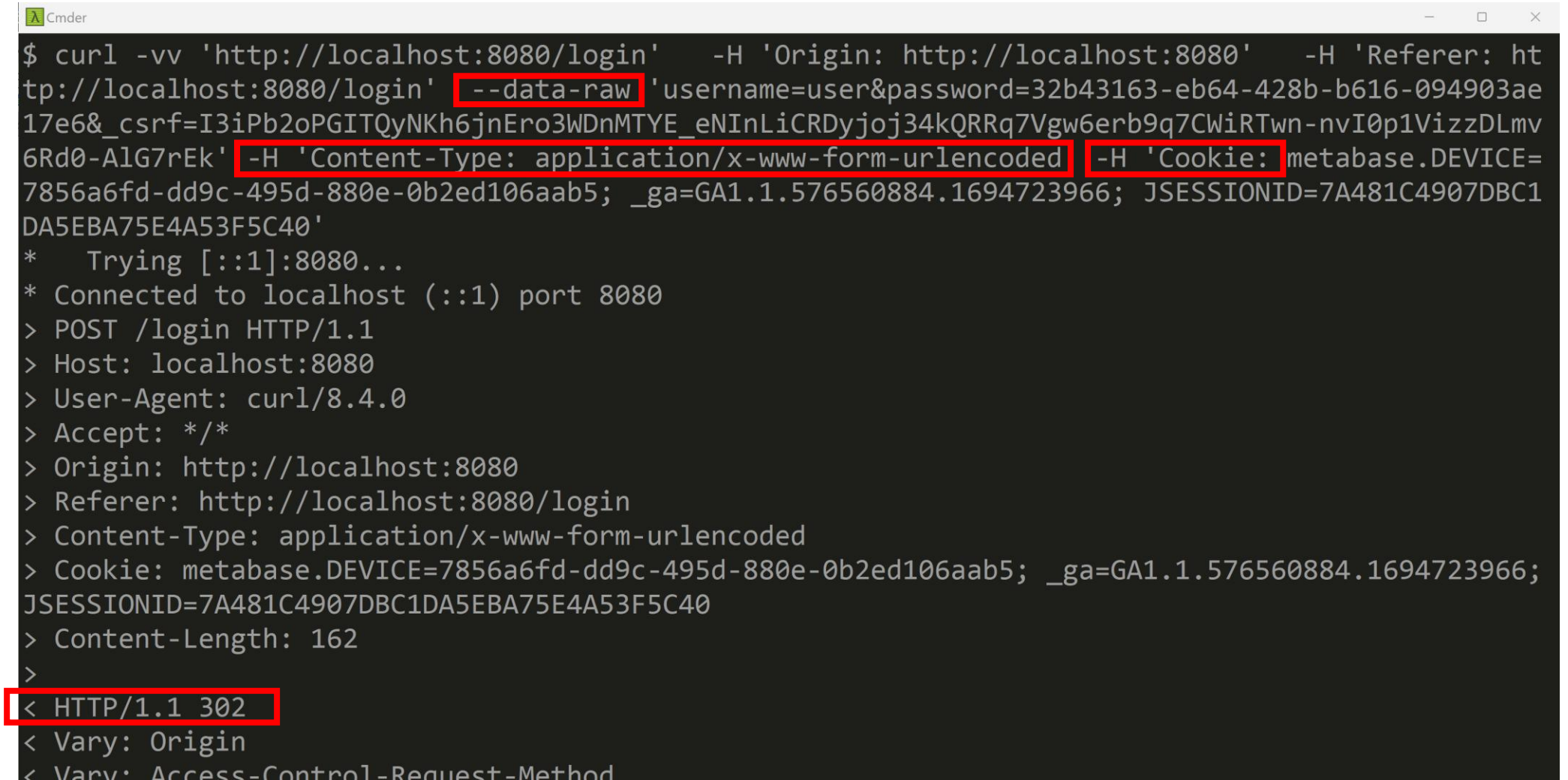
POST … payload
user,password in CLEAR text FormData



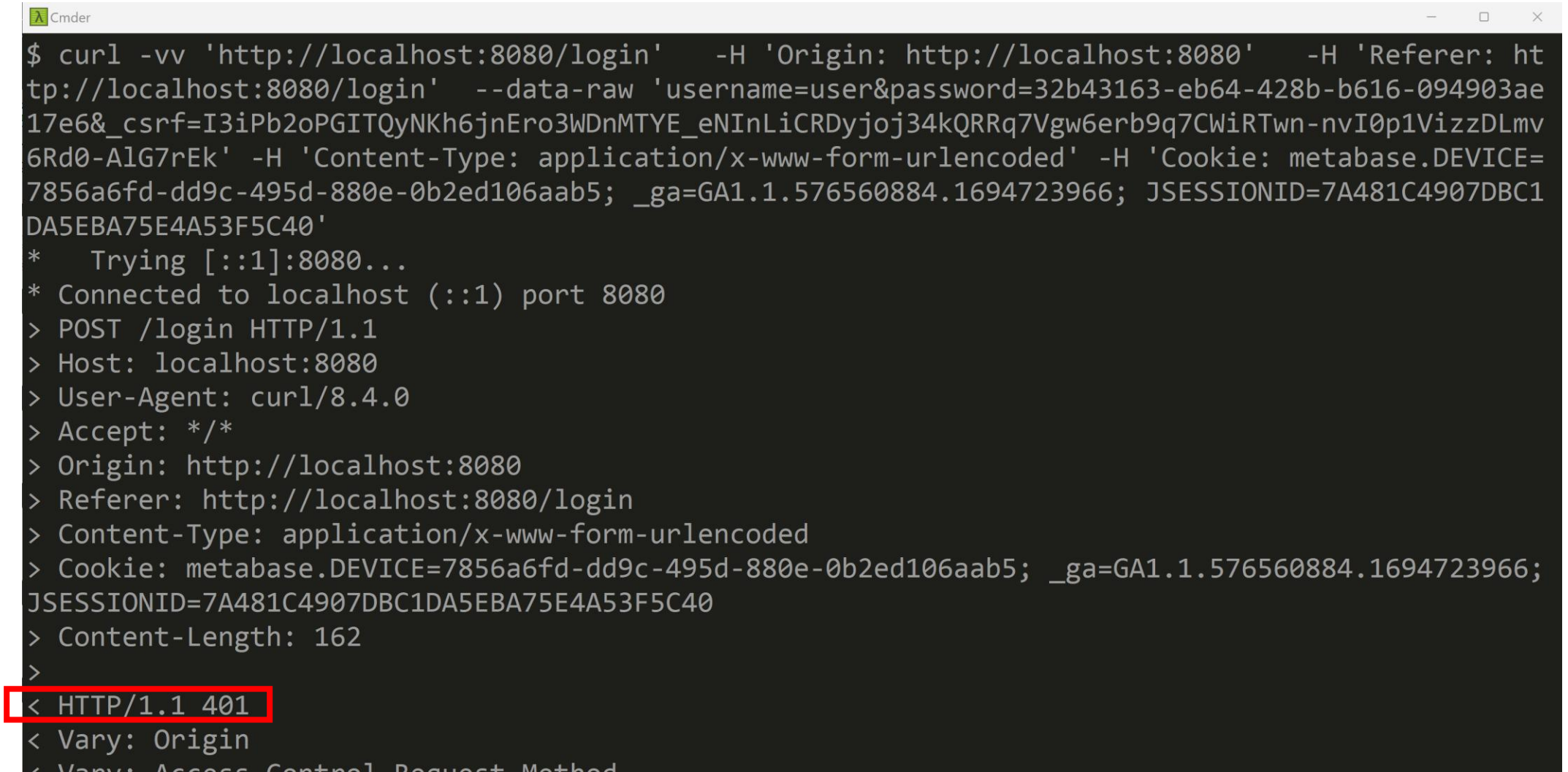
POST … payload
view parsed or view source



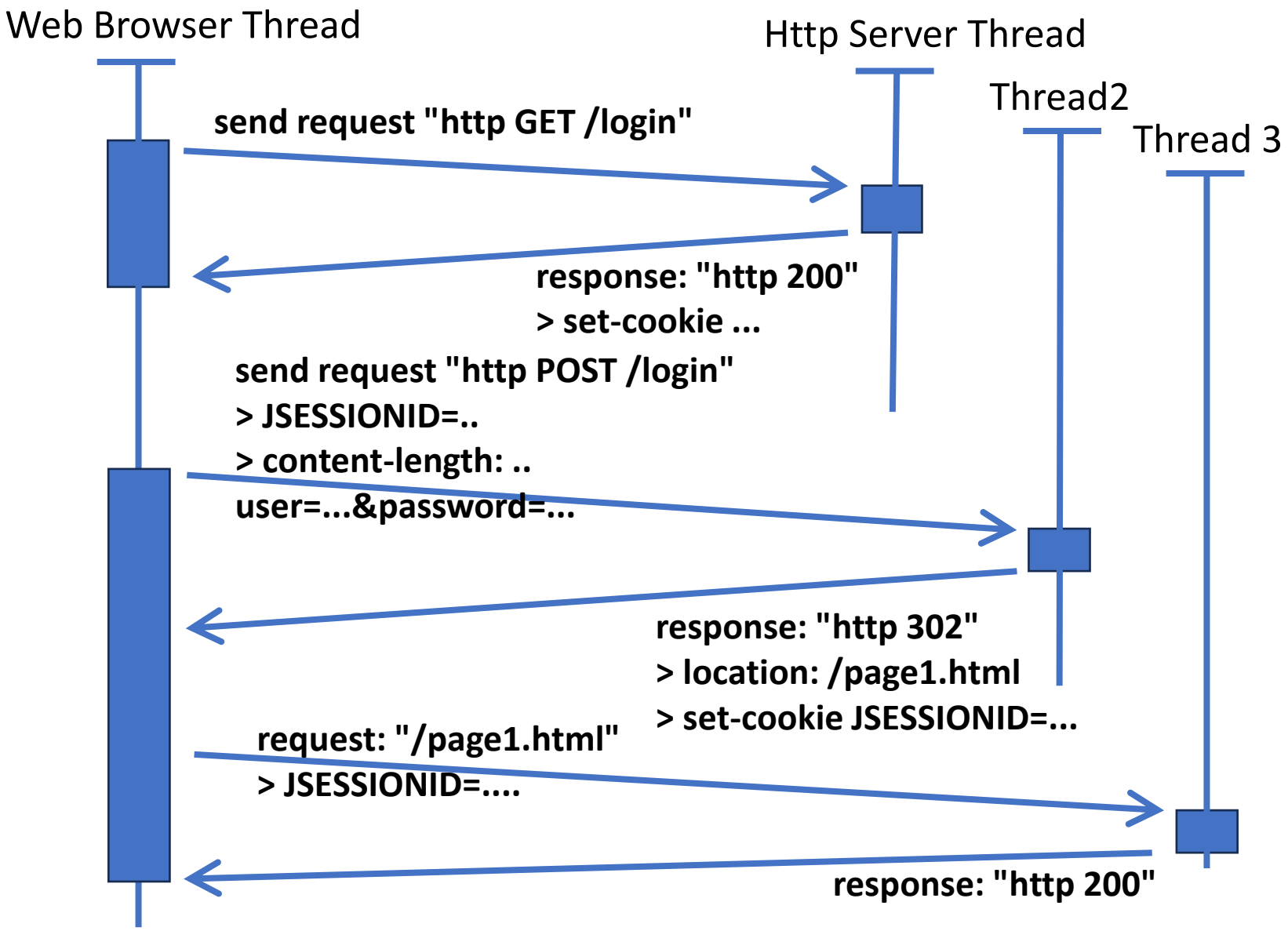
curl equivalent to http POST /login



replaying EXACT SAME curl command twice => http 401



Http POST Sequence



How did server remembered to redirect back from "/login" to "/page1.html" ?

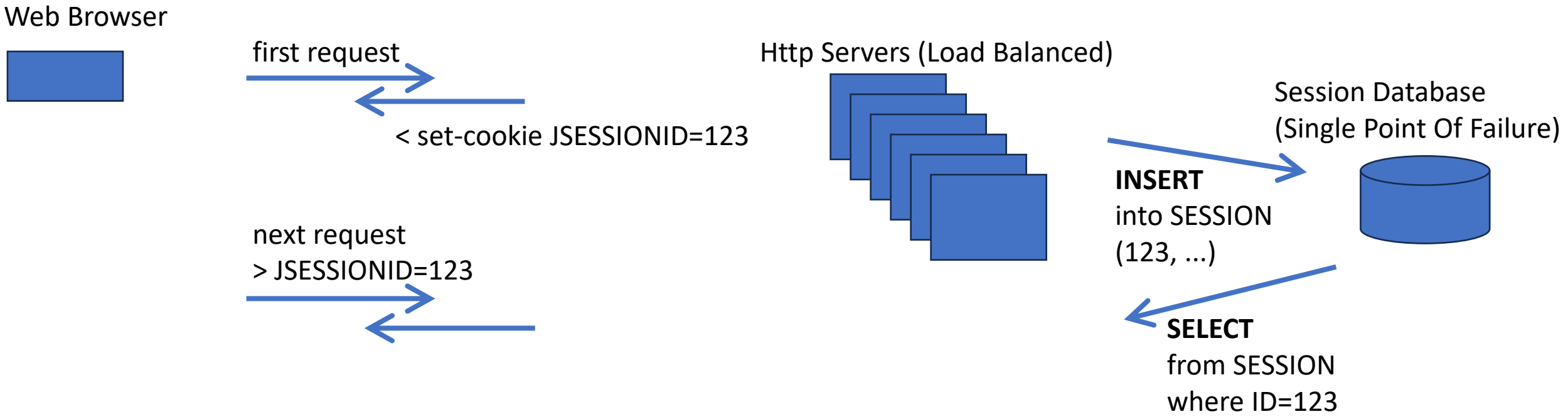
Here, server is Statefull ...

JSESSIONID are returned as "set-cookie" header from server to client

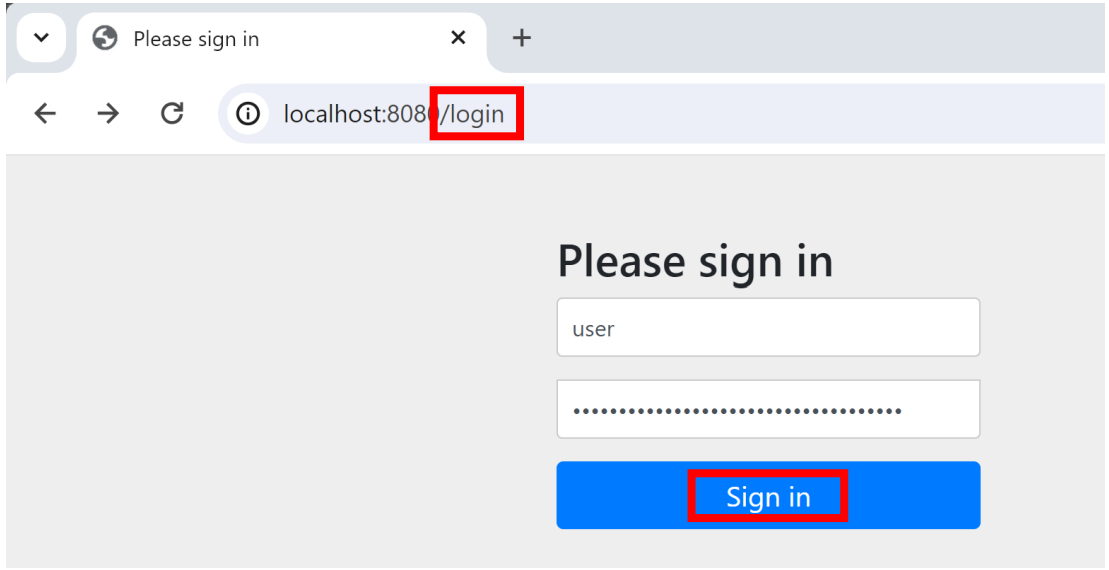
then passed back as header from client to server

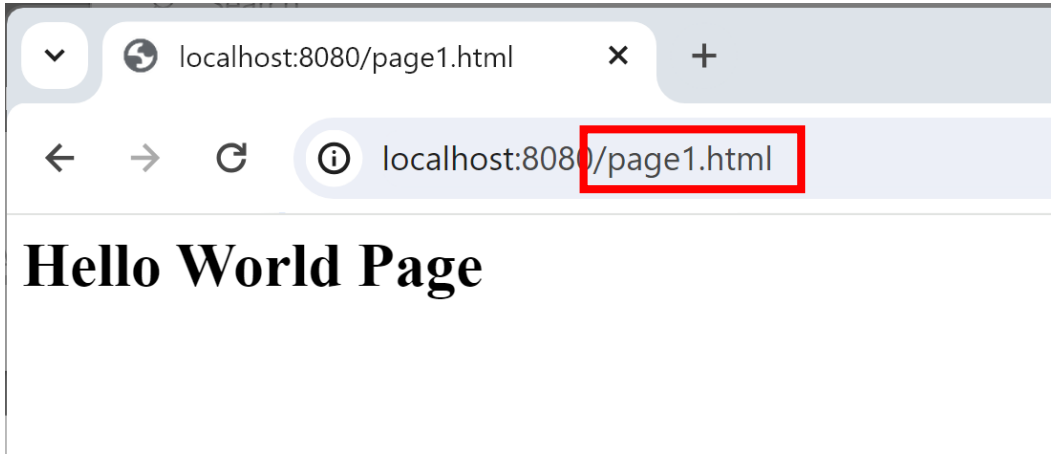
server lookup session in "database" (in memory) from JSESSIONID

Why is it "BAD" to use Cookie SESSIONID ? Statefull Server / Database ... SPOF

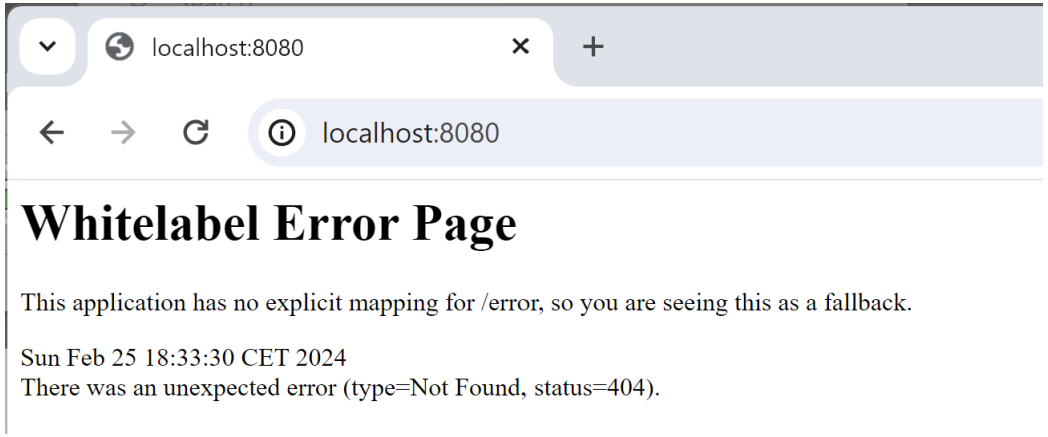


Click "Sign in" ... redirected to "/page1.html"





Browser Previous Page then Redo "Signin"
=> no redirect
"/" Not Found .. 404



Detailed Analysis of 4 http scenarios

Scenario 1/4 : No authentication

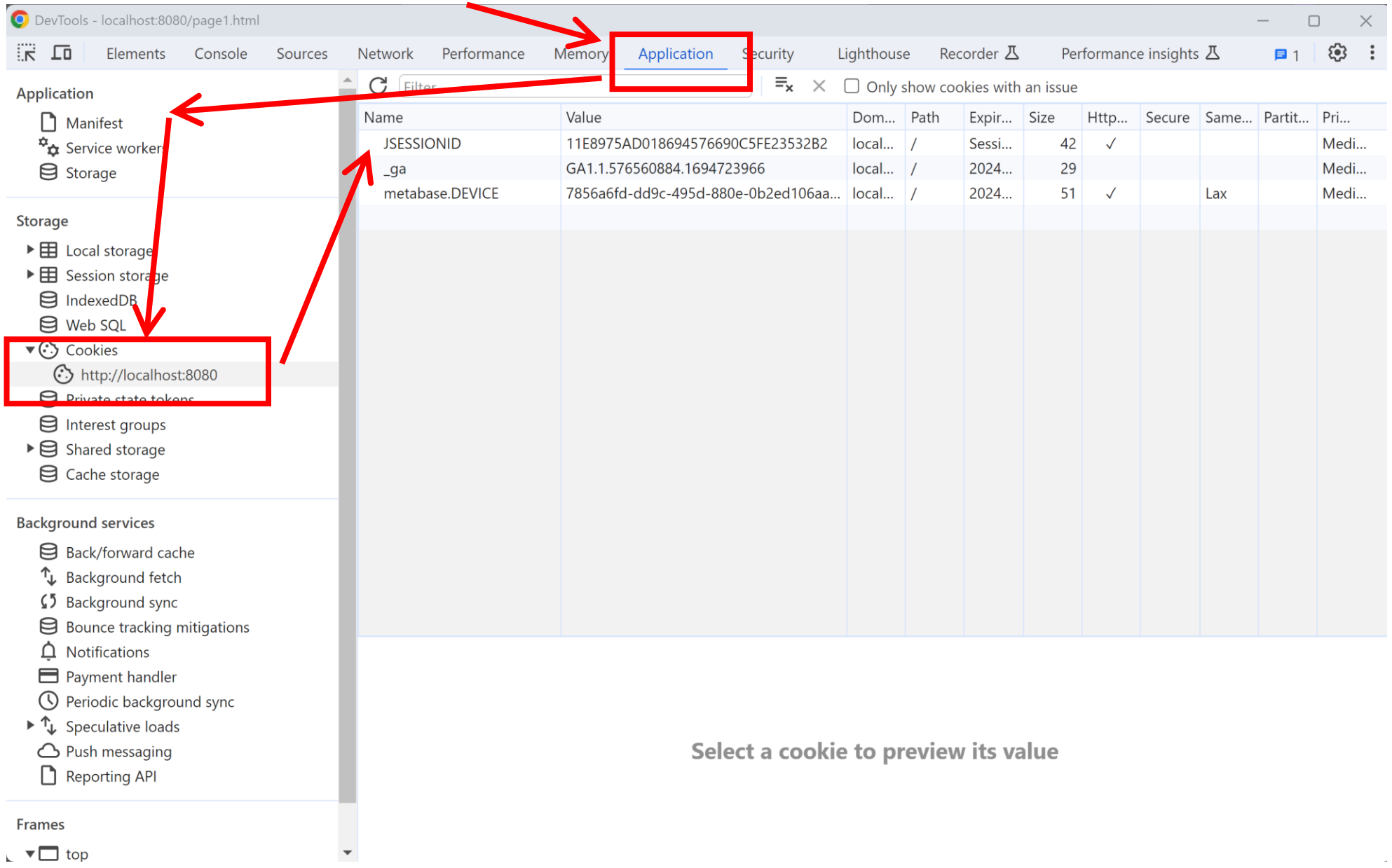
Scenario 2/4 : BAD user/password

Scenario 3/4 : user/password OK => auth success, set Cookie

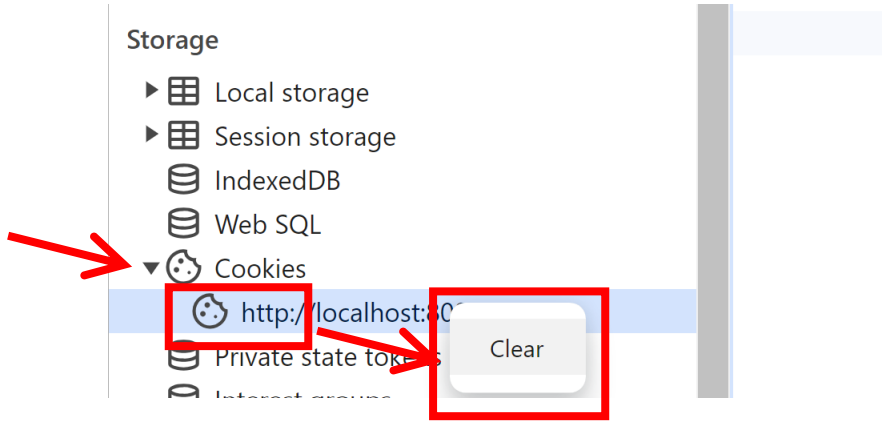


Scenario 4/4 : clear cookie

Cookies in Browser - Developer Tools



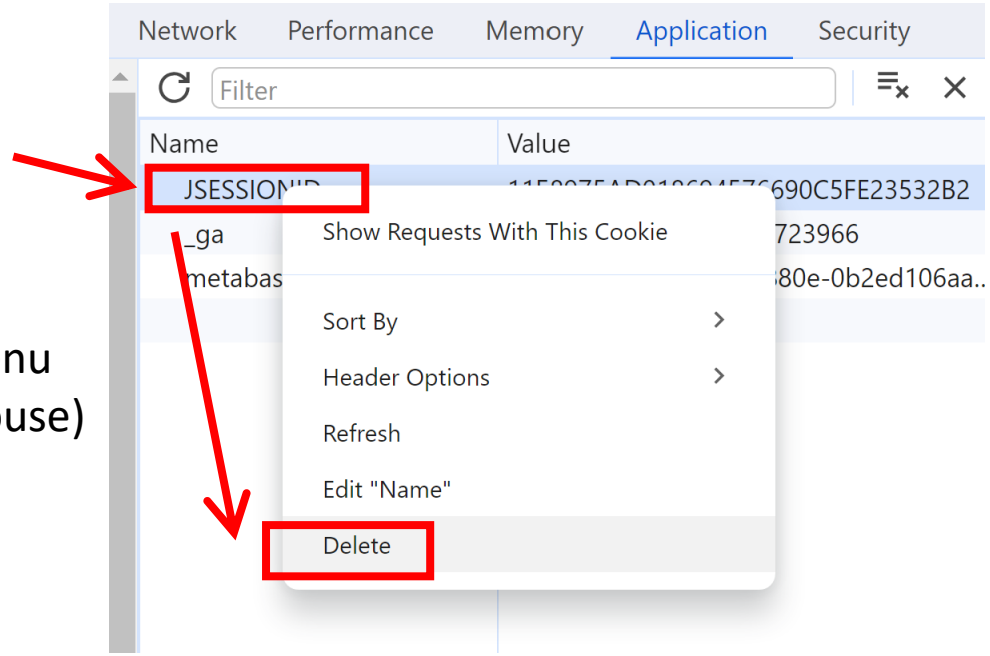
Delete Cookies [1/3]
clear all Cookies of given URL site



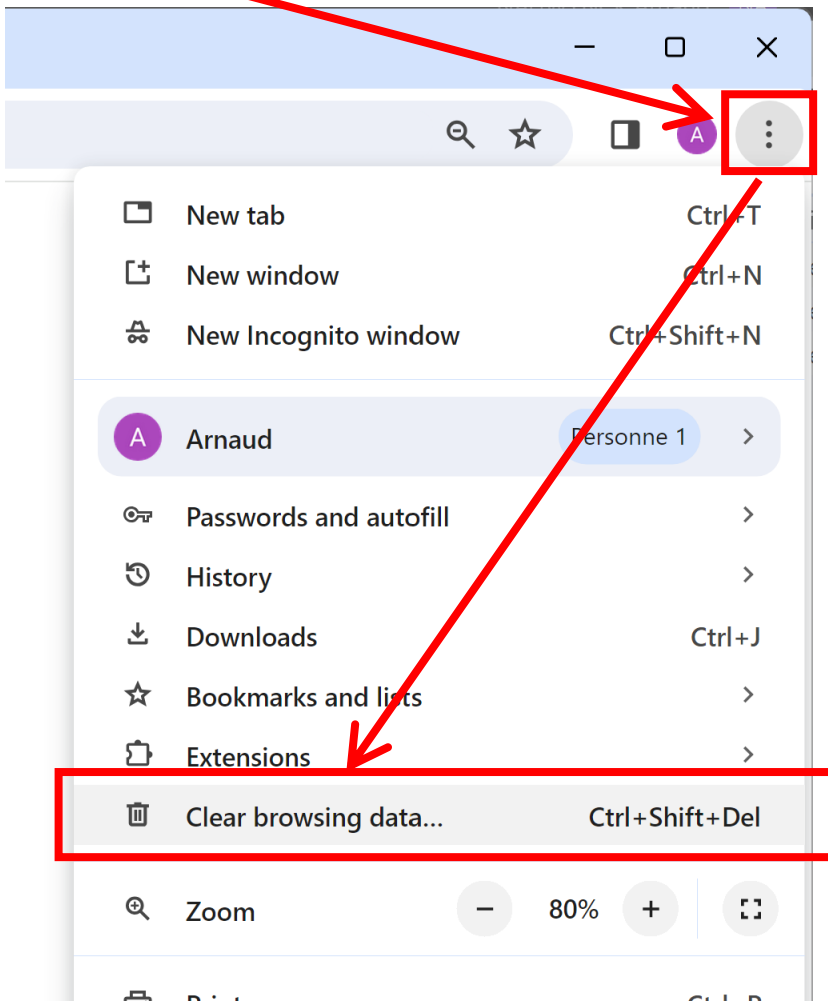
Contextual Menu
(Right click Mouse)

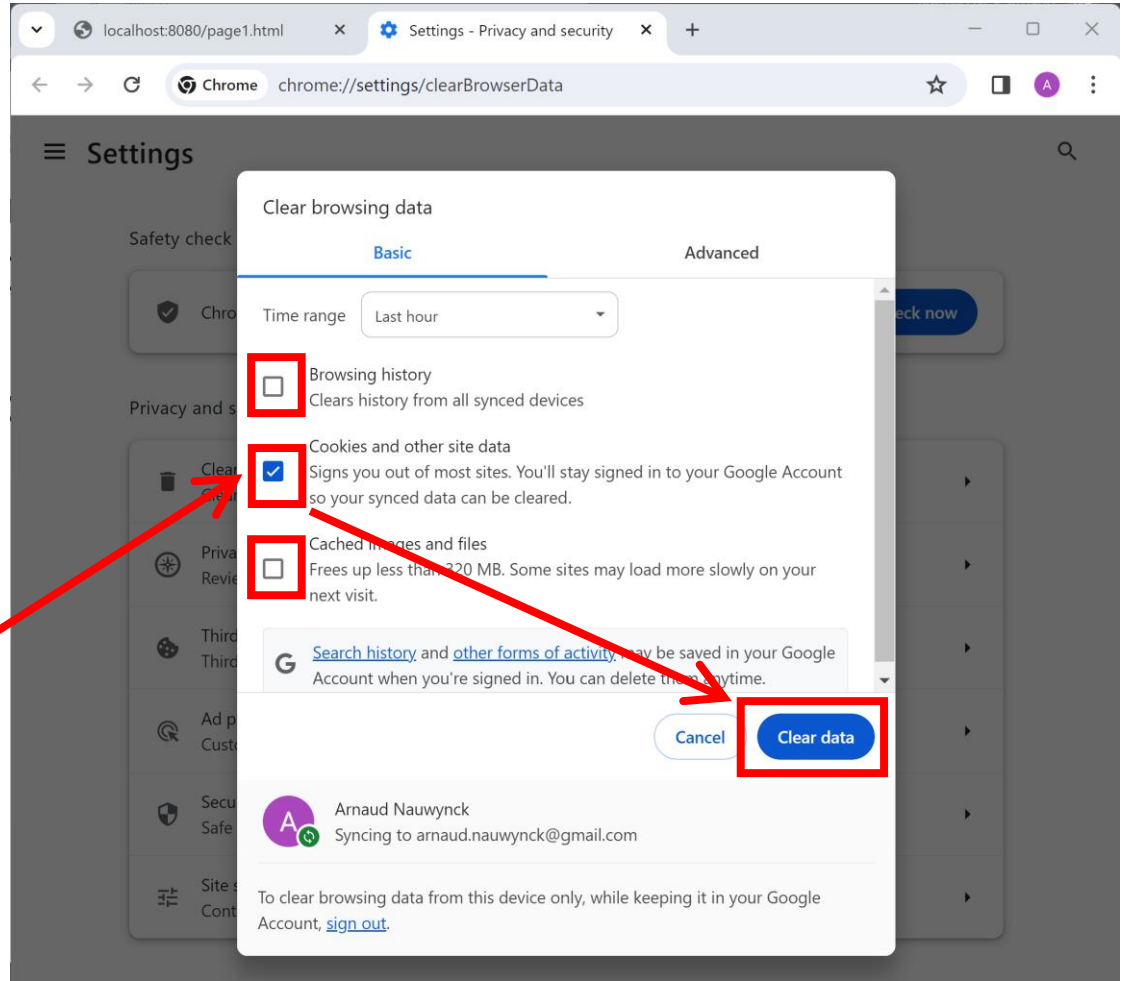
Delete Cookies [2/3]
Delete explicitly a single Site-Cookie

Contextual Menu
(Right click Mouse)

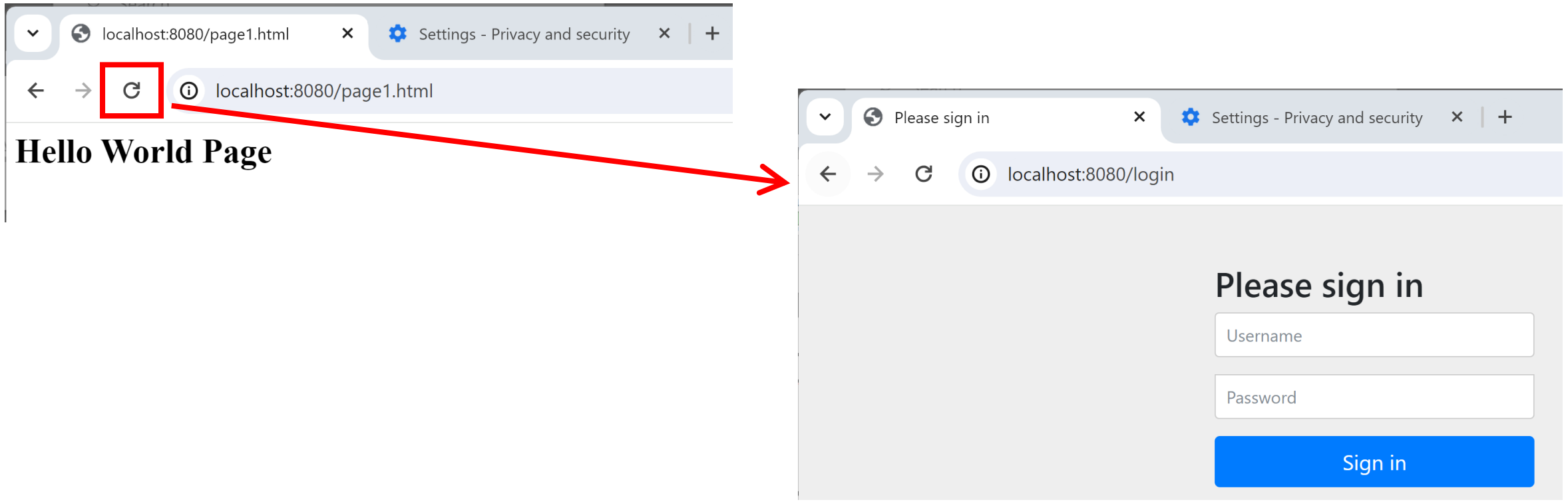


Delete Cookies [3/3]
Clear Browsing Data …
delete all Cookies (and History) of All sites





After clear "JSESSIONID" cookie
Refresh (F5) /page1.html
=> redirect again to /login



Is there a Better way (Stateless, without cookie) to redirect back from "/login" to "/page1.html" ?

yes, using http **URL query parameter**

http://localhost:8080/login**?redirect=page1.html**

… but NON replay & CSRF JSESSIONID are more secure for /login

You

why do springboot web security use set-cookie JSESSIONID in the /login html page ?

ChatGPT

In Spring Boot applications that use Spring Security for web security, the `JSESSIONID` cookie is typically set in the `/login` HTML page for session management purposes. Here's why it's done:

1. **Session Management**: When a user accesses the login page of a Spring Boot application, the server typically creates a session for that user. This session is used to maintain stateful information about the user's interaction with the application throughout their session.
2. **Authentication Flow**: In a typical authentication flow, when a user submits their credentials (username and password) via the login form, Spring Security processes these credentials and validates them. If the credentials are valid, Spring Security establishes an authenticated session for the user.
3. **Session Tracking**: The `JSESSIONID` cookie is a mechanism used by the server to track the user's session. It contains a unique identifier (the session ID) that associates the user's subsequent requests with their session data stored on the server.
4. **Secure Communication**: By setting the `JSESSIONID` cookie in the `/login` HTML page, the server ensures that subsequent requests, including the form submission with the user's credentials, are associated with the correct session. This helps maintain the integrity and security of the authentication process.
5. **Cross-Site Request Forgery (CSRF) Protection**: Additionally, setting the `JSESSIONID` cookie in the login page can also help mitigate CSRF attacks. When combined with CSRF tokens, it adds an extra layer of protection by ensuring that the session associated with the user's authentication is established before any sensitive actions, such as form submissions, are performed.

Overall, setting the `JSESSIONID` cookie in the `/login` HTML page is a standard practice in Spring Security for managing user sessions and ensuring the security of the authentication process.

Other ways to avoid "Basic" user:password credentials ?

using **Dynamic Challenges instead of static Credentials**

using **2FA or MFA (Multi Factor Authentication)**

using **OAuth2**

... then once authenticated, **using JWT Token**

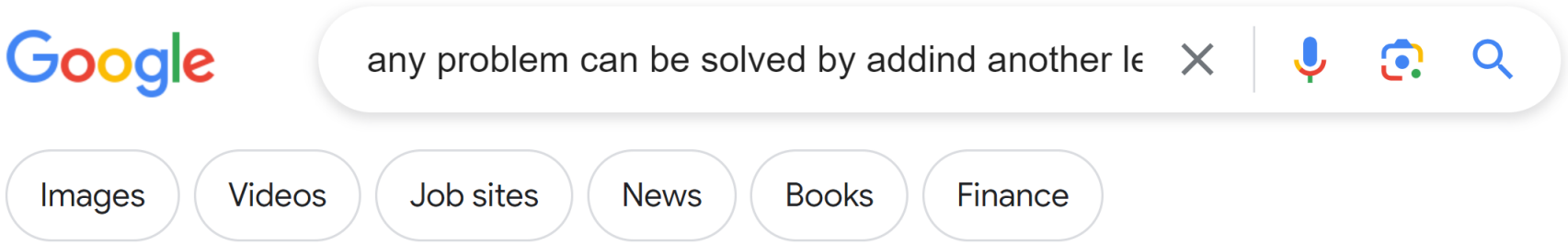
(out of scope for this basic course)

OAuth2 Standard

one password to rule them all

... OAuth2 = internally using an extra indirection level
(but still another user:password at final step)

Using Another level of Indirection…



About 1,270,000 results (0.33 seconds)

Showing results for any problem can be solved by ***adding*** another level of indirection
Search instead for any problem can be solved by addind another level of indirection

A famous aphorism of Butler Lampson goes: "All problems in computer science can be solved by another level of indirection" (the "fundamental theorem of software engineering"). This is often deliberately mis-quoted with "abstraction layer" substituted for "level of indirection".

Wikipedia
https://en.wikipedia.org › wiki › Indirection

Indirection - Wikipedia

Summary

only a very "short" introduction to "Basic Authentication for Web Apps"
using Http status code + Http Headers (Cookie, redirect Location, ...)

much more to discover:
- httpS (TLS) & Certificates, using Public Key Cryptography
- password Hashes, Managed Identities, KeyVault
- CORS, CSRF
- OAuth2
- Authorizations
- Containers, Sql Injection, VPN, Firewall, DMZ, Dos Attacks, Virus, etc, ...

Questions ?

arnaud.nauwynck@gmail.com

APPENDIX

Remark on "_csrf" hidden field in /login html page

Appendix: Csrf [1/5]

ChatGPT 3.5

You

in springboot login html page, explain the input field "_csrf"

ChatGPT

In a Spring Boot login HTML page, the input field "_csrf" is a hidden field used for CSRF (Cross-Site Request Forgery) protection. CSRF attacks involve an attacker tricking a user into unintentionally performing actions on a web application that they are authenticated with. This hidden field helps to prevent such attacks.

Here's how it typically works:

1. When a user requests the login page, the server generates a unique token and stores it in the user's session.
2. This token is then included as the value of the "_csrf" input field in the login form. However, since it's hidden, the user won't see it or interact with it directly.
3. When the user submits the login form, the CSRF token is sent along with the other form data to the server.
4. Upon receiving the request, the server compares the CSRF token submitted with the one stored in the user's session. If they match, the request is considered legitimate, and the login process proceeds.
5. If the CSRF tokens don't match, the server can reject the request, assuming it might be a CSRF attack, thus protecting the user's session from being hijacked.

This mechanism helps to ensure that the form submission originates from the legitimate user and not from a malicious website or script attempting to perform unauthorized actions on the user's behalf.

Appendix: Csrf [2/5]

ChatGPT 3.5

You

suppose that there is NO _csrf field checked, can you show me an attack

ChatGPT

Sure, I can describe a potential CSRF attack scenario without the presence of a CSRF token (_csrf) field.

Let's say you have a Spring Boot login form without CSRF protection, and the form looks like this:

html Copy code

```
<form action="/login" method="post">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username"><br><br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password"><br><br>
    <input type="submit" value="Login">
</form>
```

An attacker could create a malicious website with the following HTML:

Appendix: Csrf [3/5]

An attacker could create a malicious website with the following HTML:

html Copy code

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Malicious Site</title>
</head>
<body>
    <h1>Click the Button!</h1>
    <form action="http://victim-site.com/login" method="post">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" value="attacker"><br><br>
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" value="password"><br><br
        <input type="submit" value="Login">
    </form>
    <script>
        document.forms[0].submit(); // Automatically submits the form
    </script>
</body>
</html>
```

Appendix: Csrf [4/5]

Now, if a user who is authenticated on your application visits the attacker's website while still logged in, the form on the attacker's website will automatically submit, sending a POST request to your application's login endpoint (/login) with the user's credentials. Since there's no CSRF token validation, your application will accept the request, thinking it's a legitimate login attempt by the user.

As a result, the attacker can effectively trick the user into unintentionally performing actions on your application, such as logging in with their credentials without their knowledge or consent. This is the essence of a CSRF attack.



Appendix: Csrf [5/5]

You

so in which attacks can it occurs ?

ChatGPT

CSRF (Cross-Site Request Forgery) attacks can occur in various scenarios where an attacker can manipulate a user's web browser to perform unintended actions on a different website where the user is authenticated. Here are some common scenarios where CSRF attacks can occur:

1. **Form Submissions:** CSRF attacks often involve crafting malicious forms on attacker-controlled websites. When a user who is authenticated on another site visits the attacker's site, the form submission can trigger actions on the authenticated site.
2. **Clickjacking:** In clickjacking attacks, attackers overlay transparent or opaque elements on top of legitimate buttons or links. When a user clicks on what appears to be a harmless element, they inadvertently trigger actions on a different site where they are authenticated.
3. **Image Tags:** Attackers can embed images in web pages with image tags (<img>) that have URLs pointing to actions on a different site. When a user loads the page containing the image, the browser sends requests to the specified URLs, potentially executing unauthorized actions.
4. **Script Injections:** If an attacker can inject malicious scripts into a vulnerable website, these scripts can then manipulate the user's browser to perform actions on other sites where the user is authenticated. This often occurs in the context of Cross-Site Scripting (XSS) vulnerabilities.
5. **Malicious Links:** Attackers can distribute malicious links via email, social media, or other channels. When a user clicks on such a link while authenticated on another site, the browser can execute actions on the authenticated site without the user's consent.
6. **Embedded Content:** Content embedded in iframes or other embedded elements can execute CSRF attacks if it contains scripts or forms that trigger actions on authenticated sites when loaded by a user's browser.

These are just a few examples of how CSRF attacks can occur. The common theme is that the attacker exploits the trust established between the user and the authenticated website to perform

Message ChatGPT...